

BENUTZERHANDBUCH

Asterism

& AI Agents Map

Deine KI-Agenten, lokalen Modelle, MCP-Server und Skills auf einer Karte

Asterism 0.10.1 · AI Agents Map 0.22.0

Oktober 2026

Über dieses Handbuch

Dieses Handbuch beschreibt eine Anwendung, die es in zwei Ausgaben gibt:

Ausgabe	Asterism	AI Agents Map
Was es ist	Eine eigenständige GTK 4-/libadwaita-App für jeden Linux-Desktop	Ein Plugin für den Omarchy-Desktop, geöffnet über einen Knopf in der Leiste
Installation aus	AUR, AppImage oder Quellcode	Dem Omarchy Plugin Marketplace
Aussehen	Folgt dem hellen/dunklen Modus und der Akzentfarbe des Systems	Farben des Omarchy-Themes oder GNOMEs Adwaita-Aussehen
Einstellungsordner	`~/.config/asterism/`	`~/.config/omarchy/ai-agents-map/`

Alles andere — Setup, die Karte, Verbinden durch Zeichnen, lokale LLMs, Modelle, MCP-Server und Skills — ist in beiden gleich. Was nur für eine Ausgabe gilt, ist mit ASTERISM oder OMARCHY markiert.

Schreibweisen

Hinzufügen	Ein Knopf, Reiter oder Menüeintrag im Fenster.
Einstellungen › **Aussehen**	Ein Weg durch die Oberfläche: Einstellungen öffnen, dann **Aussehen**.
`~/.agents/skills`	Eine Datei, ein Ordner oder ein Befehl. `~` ist dein persönlicher Ordner.
Strg + ,	Ein Tastenkürzel.

> **HINWEIS** Hinweise geben nützlichen Hintergrund.

> **TIPP** Tipps zeigen einen schnelleren Weg oder eine gute Gewohnheit.

> **VORSICHT** Vorsicht-Kästen weisen auf etwas hin, das schwer rückgängig zu machen ist.

Inhalt

1 Einführung 1
1.1 Was es kann 1
1.2 Grundbegriffe 2
1.3 Nichts ändert sich ohne Vorschau 2
2 Installation 3
2.1 Asterism 3
2.2 AI Agents Map auf Omarchy 3
2.3 Erster Start 3
2.4 Die App entfernen 4
3 Das Fenster im Überblick 5
3.1 Setup 6
3.2 Auswählen, was die Karte zeigt 8
3.3 Mit der Karte arbeiten 8
3.4 Offene Probleme 10
3.5 Updates und Vorschläge 10
4 Agenten 11
4.1 Details eines Agenten 11
4.2 Agenten starten und stoppen 12
4.3 Netzwerkverkehr 12
4.4 Subagenten 13
4.5 Einrichten… 13
4.6 Anmelden 13
4.7 Agenten aktualisieren 14
4.8 Agenten, die die App installieren kann 14
4.9 Neue Agenten 15
5 Lokale LLMs 16
5.1 Ein lokales LLM installieren 16
5.2 Mit den Agenten starten und stoppen 16
5.3 Details eines lokalen LLMs 17
5.4 GPU-Speicher und GPU-Speicher freigeben 17
5.5 Modelle gleichzeitig 17
5.6 Von anderen Apps genutzt 18
6 Modelle 19
6.1 Von Hugging Face holen 19
6.2 Installiert 20
6.3 Kontextgröße 20
6.4 Auf diesem PC 20
6.5 Modelle verschieben und löschen 20
6.6 Ein Modellordner 20
7 Verbinden durch Zeichnen 22
7.1 Agent ↔ lokales LLM 22
7.2 Agent ↔ MCP-Server 22
7.3 Agent ↔ Skill-Ordner 22
7.4 Wenn eine Verbindung fehlschlägt 22
8 MCP-Server 23
8.1 Woher Server kommen 23
8.2 Einen Server einrichten 23
8.3 Server verwalten 24
9 Skills 25
9.1 Skills finden 25
9.2 Skill-Ordner 25
10 Einstellungen und Hilfe 26
10.1 Einstellungen 26
10.2 Hilfe 27
10.3 Speicherplatz 27
11 Smart Check 29
11.1 Zusammenfassung des PC-Setups 30
11.2 Gefundene Updates 30
11.3 Token-Statistik 30
12 Gesundheits- und Sicherheitscheck 31
12.1 Die Karten lesen 31
12.2 Eine Einstellung ändern 32
12.3 Was bei jedem Agenten geprüft wird 32
12.4 Das Risiko steuern 33
12.5 Die Begriffe erklärt 34
13 Add-ons 36
13.1 Asterism einrichten 36
13.2 Die Add-on-Seite 37
13.3 Installieren, ein- und ausschalten, aktualisieren, entfernen 38
13.4 Datenschutzstufen 39
13.5 Arten von Add-ons 39
13.6 Ein neuer Agent auf deinem Computer 40
13.7 Ein Add-on erstellen 40
14 Datenschutz und Sicherheit 41
15 Problemlösung 42
A Referenz 43
A.1 Tastenkürzel 43
A.2 Dateien und Ordner 43
A.3 Glossar 44

KAPITEL 1

Einführung

Asterism zeigt das KI-Setup deines Computers als Karten und als Karte: welche Coding-Agenten installiert sind, welche Sprachmodelle sie nutzen — Cloud-Dienste und lokale Laufzeiten —, welche Modelle diese bereitstellen und welche MCP-Server und Skills jeder Agent hat. Und es lässt dich dieses Setup durch Linienzeichnen ändern.

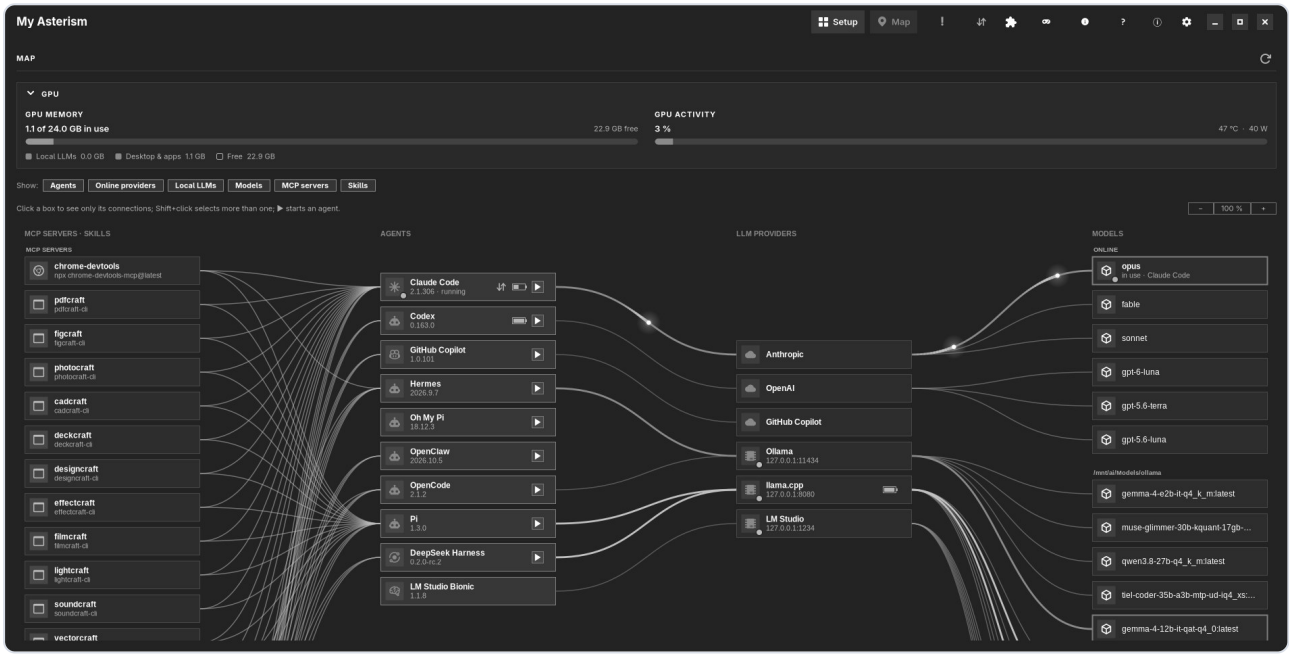


Abbildung 1 · Das Hauptfenster: die Karte über die ganze Breite, darüber die GPU-Karte, in der Kopfzeile das ! für offene Probleme.

1.1 Was es kann

- **Coding-Agenten** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes und mehr: ihre Version, ihr Abo, ihre Limits und ihre Nutzung, und was jedem noch fehlt, mit dem Befehl, der es behebt.
- **Lokale LLMs** — llama.cpp, Ollama, LM Studio, vLLM, Jan und KoboldCpp: installieren, Modelle herunterladen, die zur Grafikkarte passende Kontextgröße einstellen und sie zusammen mit den Agenten starten, die sie nutzen.
- **MCP-Server und Skills** — gefunden auf deinem Computer oder im Web, mit empfohlenen Servern und einer Suche in der offiziellen MCP-Registry.
- **Offene Probleme** — ein ! in der Kopfzeile, wenn etwas zu beheben ist, jedes Problem mit dem Weg dorthin, wo es behoben wird, und wofür dein Setup taugt.

1.2 Grundbegriffe

Agent	Ein Programm, mit dem du sprichst und das Code schreibt oder Aufgaben erledigt, etwa Claude Code.
LLM-Anbieter	Wo das Modell eines Agenten läuft: ein Cloud-Dienst (Anthropic, OpenAI, …) oder eine lokale LLM-Laufzeit auf diesem Computer (llama.cpp, Ollama, …).
Modell	Das Sprachmodell selbst. Lokale Modelle sind Dateien, meist im GGUF-Format.
MCP-Server	Ein kleines Programm oder ein Webdienst, der Agenten zusätzliche Werkzeuge gibt — Dateien lesen, im Web suchen, einen Browser steuern. MCP ist das Model Context Protocol.
Skill	Ein Ordner mit Anweisungen (einer `SKILL.md`) und manchmal Skripten, die einem Agenten eine bestimmte Aufgabe beibringen.

1.3 Nichts ändert sich ohne Vorschau

Jede Änderung an den Einstellungen eines Agenten wird zuerst Zeile für Zeile gezeigt, bevor etwas geschrieben wird. Die alte Datei bleibt als Sicherung erhalten, und die Leiste, die nach der Änderung erscheint, hat **Rückgängig**. Die App selbst läuft nie mit Administratorrechten; nur das Installieren eines Agenten, der als Systempaket kommt, fragt nach deinem Passwort, in einem Terminal, das du siehst.

KAPITEL 2

Installation

2.1 Asterism ASTERISM

Wähle einen dieser Wege:

- **Arch Linux (AUR):** installiere das Paket `asterism` mit deinem AUR-Helfer.
- **AppImage:** lade `Asterism-<Version>-x86_64.AppImage` von der Release-Seite unter `github.com/uBruckhaus/Asterism` herunter, mache es ausführbar und starte es. Das AppImage nutzt das Python 3 deiner Distribution mit PyGObject, GTK 4 und libadwaita:

Fedora	`sudo dnf install python3-gobject gtk4 libadwaita`
Ubuntu / Debian	`sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1`
Arch	`sudo pacman -S python-gobject gtk4 libadwaita`

- **Aus dem Quellcode:** starte `bin/asterism` in einem Klon des Repositorys.

Die App bringt ihre eigenen Symbole mit, eine Symbolschrift ist nicht nötig. Um Agenten im Fenster laufen zu lassen, installiere außerdem VTE für GTK 4 (`vte4`).

2.2 AI Agents Map auf Omarchy OMARCHY

Installiere das Plugin aus dem Omarchy Plugin Marketplace oder in einem Terminal:

```
omarchy plugin add https://github.com/uBruckhaus/omarchy-ai-agents-map.git --enable
```

In der Leiste erscheint ein Roboter-Knopf. Alles andere, was es braucht, bringt eine normale Omarchy-Installation mit. Um Agenten im Fenster laufen zu lassen, installiere VTE: `sudo pacman -S vte4` .

2.3 Erster Start

Beim ersten Öffnen fragt die App, ob sie sich in den App-Starter eintragen soll. Dort wird nichts geschrieben, solange du nicht zustimmst; ändern kannst du es später unter Einstellungen › **Im App-Starter**.

Das Fenster ist ein normales App-Fenster: Es kann gekachelt, schwebend, auf einen anderen Arbeitsbereich verschoben und mit `Alt`+`Tab` erreicht werden. ASTERISM Ein neues Asterism öffnet sich auf seiner **Willkommensseite**: Die auf diesem Computer installierten Agenten und lokalen LLMs sind Karten, die schon angehakt sind, und **Einschalten** bringt sie auf die Karte (siehe Asterism einrichten). OMARCHY Der Leisten-Knopf öffnet das Fenster, holt es nach vorn oder versteckt es, wenn es schon vorn ist. Schließen versteckt das Fenster; das Programm läuft im Hintergrund weiter, damit es sofort wieder aufgeht.

> **HINWEIS** Läuft ein Agent im Fenster, fragt das Schließen zuerst: **Stoppen und schließen** (dann stoppt auch das lokale LLM des Agenten und gibt den Grafikspeicher frei) oder **Weiterlaufen lassen**.

2.4 Die App entfernen

OMARCHY `omarchy plugin remove ubruckhaus.ai-agents-map`. ASTERISM Entferne das Paket oder lösche das AppImage.

Schalte vorher Einstellungen › **Lokale LLMs mit ihren Agenten starten** aus, damit auch der Hintergrund-Wächter entfernt wird, und Einstellungen › **Im App-Starter**, um den Starter-Eintrag zu löschen. Was du mit der App geändert hast, bleibt: Einstellungen der Agenten, MCP-Server, Skills und installierte lokale LLMs. Deinstalliere ein lokales LLM zuerst (Rechtsklick › **Deinstallieren…**), wenn es weg soll. Um auch den eigenen Zustand der App zu löschen:

```
rm -rf ~/.config/omarchy/ai-agents-map ~/.config/asterism \
       ~/.local/state/ai-agents-map ~/.cache/ai-agents-map
```

KAPITEL 3

Das Fenster im Überblick

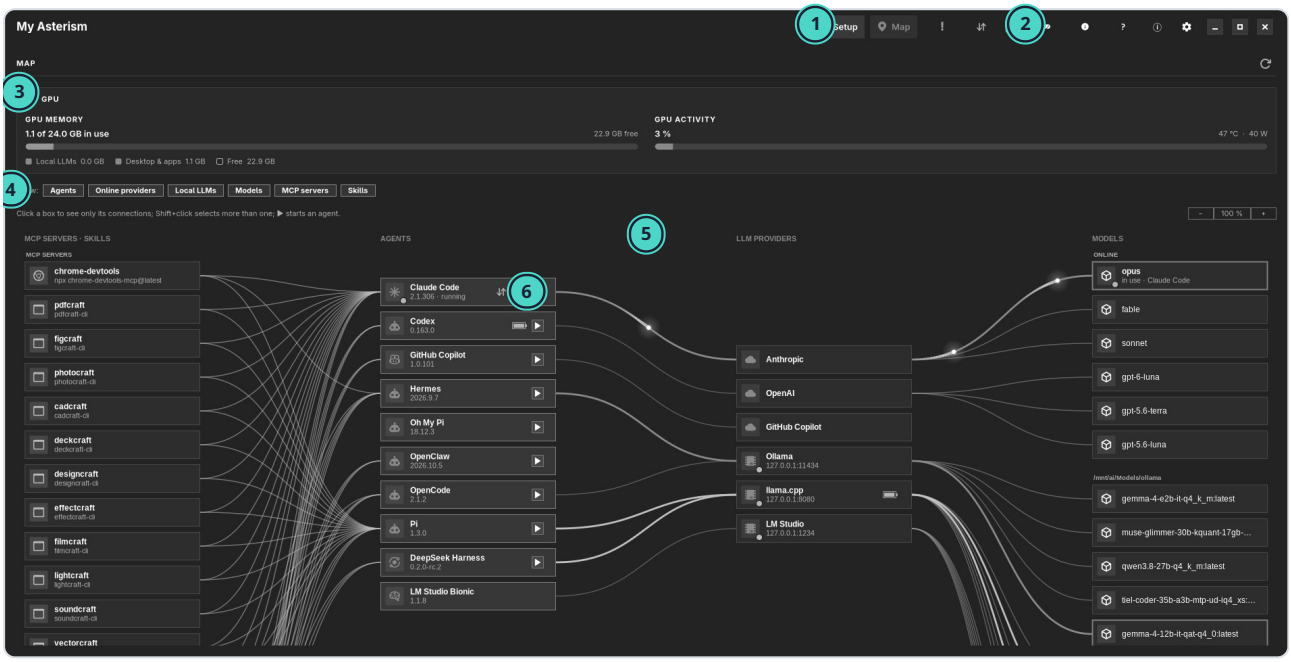


Abbildung 2 · Das Hauptfenster. Die Nummern entsprechen der Liste darunter.

1. **Setup** und **Karte** — die beiden Ansichten, auf jeder Seite; die gerade sichtbare ist hervorgehoben. Setup zeigt jeden Agenten, jedes lokale LLM, jeden MCP-Server, jeden Skill-Ordner und jedes Modell als Karte: installieren, aktualisieren, anmelden, entfernen (siehe unten).
2. Die Symbole der Kopfzeile: ↻ aktualisieren (F5: erneut nachsehen, wie beim Start), **!** offene Probleme und Updates (rot: etwas zu beheben; Akzentfarbe: nur Updates und Vorschläge; nicht da, wenn es nichts gibt), ⓘ die Legende (was jeder Punkt, jede Linie und jede Markierung der Karte bedeutet), ⇅ der Netzwerkverkehr der laufenden Agenten (siehe Netzwerkverkehr), die zwei Funkelsterne für **Smart Check** (nur solange Claude Code oder Codex installiert ist, siehe Smart Check), die Add-ons ASTERISM, das gefüllte **i** für Information (siehe Hilfe), **?** Hilfe (F1) und das Zahnrad für die Einstellungen (Strg + ,). Halte die linke Maustaste auf einem Symbol gedrückt und zieh es zur Seite, um es woanders hinzusetzen: Ein Balken zeigt, wo es landet, und die Reihenfolge bleibt erhalten.
3. **GPU** — GPU-Speicher und GPU-Aktivität: wie die Grafikkarte gerade genutzt wird, in einer Karte, die sich zuklappt (zugeklappt behält ihre Zeile die Zahlen).
4. **Zeigen**-Schalter — kürzen die Karte auf die Bereiche, an denen du arbeitest.
5. **Die Karte** — MCP-Server und Skills ← Agenten → LLM-Anbieter → Modelle, über die ganze Breite, unter ihrer Überschrift **KARTE**.
6. ▶ im Feld eines Agenten startet ihn in einem Terminal im Fenster; während er dort läuft, holt der Knopf sein Terminal zurück.

3.1 Setup

Die Karte zeigt nur, was eingerichtet ist: installierte Agenten und lokale LLMs, die MCP-Server und Skill-Ordner, die Agenten nutzen, und Modelle. Alles andere geschieht im **Setup** (der Knopf in der Kopfzeile neben **Karte**), eine Karte pro Eintrag, in Abschnitten, die sich mit einem Klick auf ihre Leiste zuklappen:

- **Wie etwas auf deine Karte kommt** — eine zugeklappte Karte ganz oben; öffne sie für fünf nummerierte Schritte: einschalten (oder finden), was du nutzt, installieren, was fehlt, bei einem Online-Agenten anmelden, durch Zeichnen verbinden, und was von selbst erscheint.
- **Was du damit machen kannst** — wofür das Setup taugt (Alltagshilfe, Programmieren, Bilder, Sprache, Recherche und Schreiben), je eine Kachel mit dem, was da ist, und dem nächsten Schritt; zugeklappt sagt ihre Zeile, wie viele bereit sind.
- **KI-Agenten, Lokale LLMs** — alle, die die App kennt. Eine Karte mit blauem Rahmen ist auf diesem PC installiert; eine graue nicht, und sie hat **Installieren**, wo installierte **Deinstallieren** haben. OMARCHY Ein Klick auf eine installierte Karte öffnet ihre Details. ASTERISM Ein Klick hakt die Karte an oder ab (✓), dann schaltet **Übernehmen** ihr Add-on ein oder aus. Ausschalten nimmt sie von der Karte; gelöscht wird nichts.
- **Online-Anbieter** ASTERISM — das Add-on der Online-Anbieter; sein Menü hat **Anmelden bei …** für jeden installierten Agenten, der sie nutzt.
- **MCP-Server, Skills, Modelle** — eine Karte für jeden installierten Eintrag. Ein Klick öffnet seine Details (bei einem Modell sein Menü). Die Karte mit **+** am Ende eines Abschnitts fügt einen hinzu: **MCP-Server hinzufügen**, **Skills hinzufügen**, **Modell hinzufügen**, **Add-ons holen**.
- ⋯ oder ein Rechtsklick auf eine Karte öffnet dasselbe Menü wie das Feld auf der Karte: aktualisieren, anmelden, starten oder stoppen, Details, deinstallieren oder entfernen, ein Modell verschieben.
- Ein gelbes ↑ **Version** auf einer Karte heißt, dass ein Update bereitsteht; ihr Menü hat **Aktualisieren**.
- Ein rotes **!** auf einer Karte heißt: Dort gibt es ein offenes Problem; zeige mit der Maus darauf, um zu sehen, welches.
- Eine kleine Batterie auf der Karte eines Online-Agenten zeigt, was von seinen Limits übrig ist (eine Sitzung, eine Woche): Das knappste zählt, gelb bei einem Viertel oder weniger, rot unter einem Zehntel; zeige mit der Maus darauf für jedes Limit und wann es zurückgesetzt wird. Auf der Karte eines laufenden lokalen LLMs ist die Batterie der noch freie GPU-Speicher, in denselben Farben; zeige mit der Maus darauf, um zu sehen, wie viel dieses lokale LLM belegt.
- Das Aktualisieren-Symbol (F5) in der Kopfzeile sieht erneut nach, was installiert, eingerichtet und angemeldet ist, und führt die Prüfungen eines Starts noch einmal aus (neue Apps, Add-ons, Updates, offene Probleme).

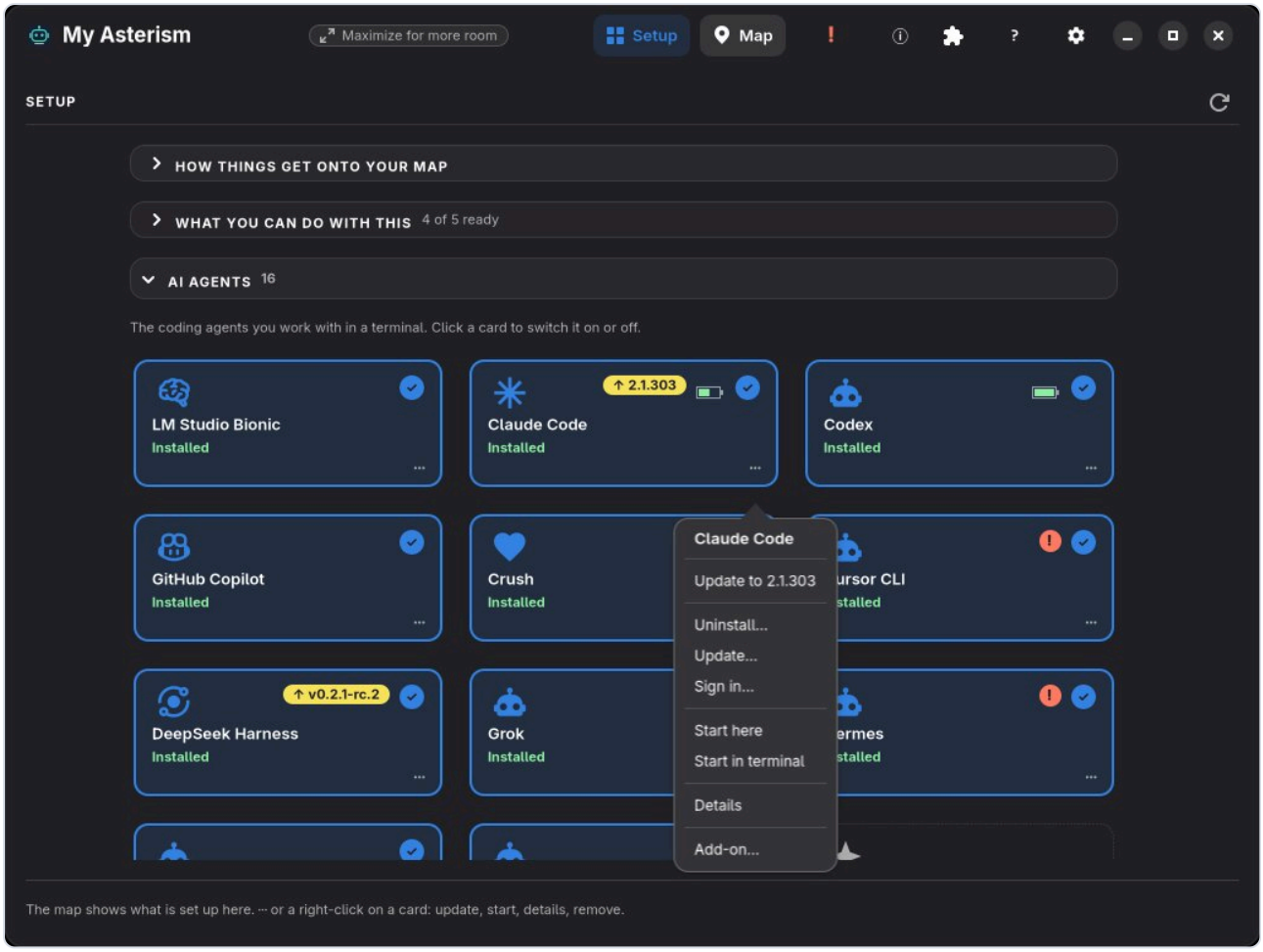


Abbildung 3 · Setup: Installierte Agenten haben einen blauen Rahmen, eingeschaltete ein ✓, Updates ein gelbes und Probleme ein rotes Abzeichen; das Menü von Claude Code.

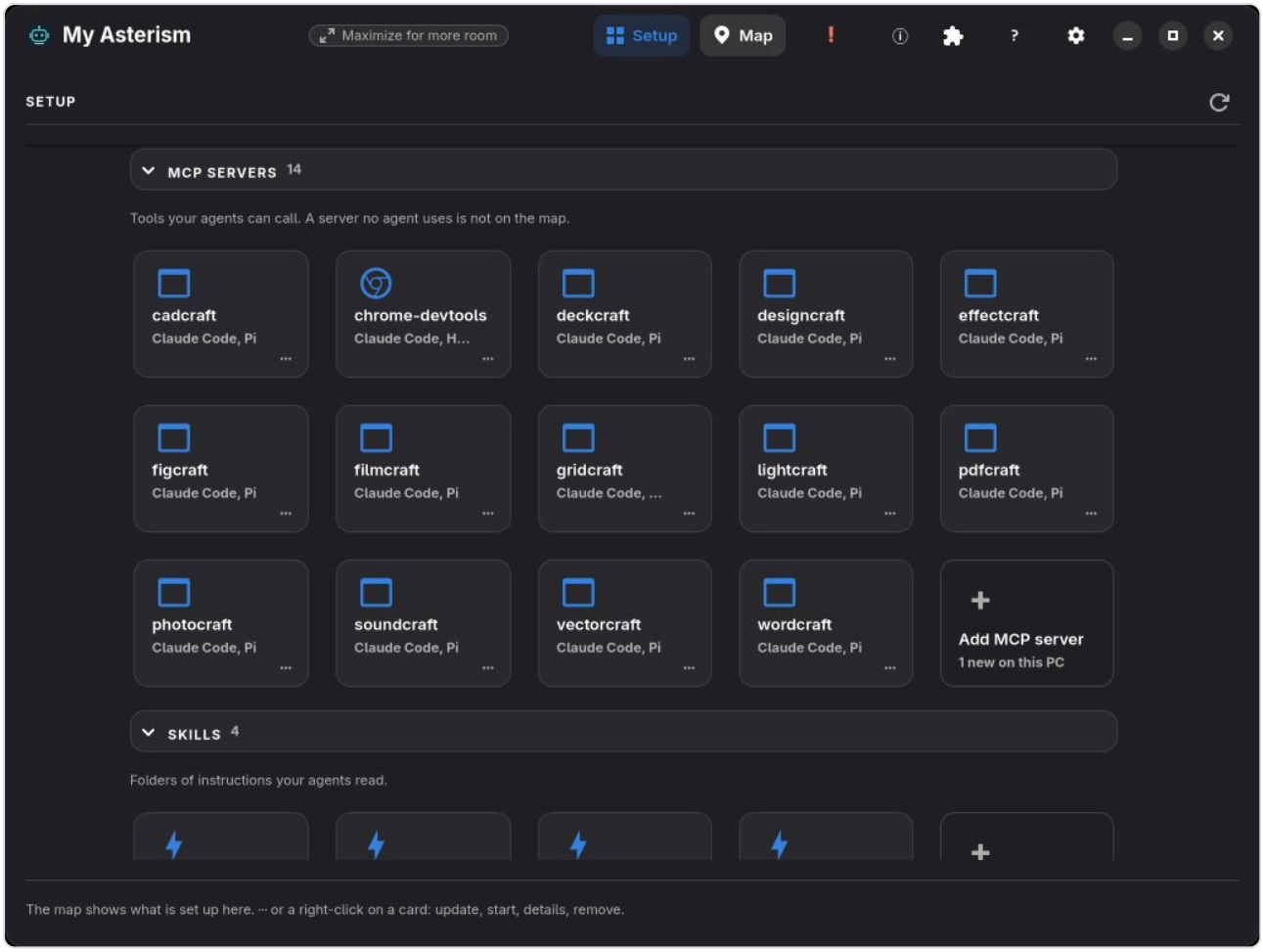


Abbildung 4 · MCP-Server und Skills, jeder Abschnitt mit einer Karte zum Hinzufügen.

Es gibt nichts zu verstecken: Was du nicht auf der Karte willst, wird hier ausgeschaltet oder entfernt. Läuft eine Installation oder eine Anmeldung in einem Terminal im Fenster, führt **Zurück** (oder **×** auf ihrem Reiter) zu der Seite zurück, auf der sie gestartet wurde.

3.2 Auswählen, was die Karte zeigt

Die Karte zeigt alles, was eingerichtet ist. Die **Zeigen**-Schalter unter der GPU-Karte lassen ganze Bereiche weg — zum Beispiel alle Skills oder die lokalen LLMs und ihre Modelle — und werden gemerkt.

In einem schmalen Fenster (halber Bildschirm oder weniger) wählt die Karte sie selbst, und eine Zeile neben den Schaltern sagt das: Solange ein Agent läuft, Agenten und Anbieter (wer über was arbeitet); solange keiner läuft, Anbieter und Modelle. Ein dort geklickter Schalter bleibt, bis sich das ändert, und wird nicht gemerkt; das Fenster in voller Größe behält deine eigene Wahl.

3.3 Mit der Karte arbeiten

- **Zeige** mit der Maus auf ein Feld, um es mit allem, womit es verbunden ist, hervorzuheben.
- **Klicke** ein Feld an, um es auszuwählen (✓); `Umschalt`+Klick fügt weitere hinzu. `Esc` oder **Auswahl aufheben** hebt sie auf.

- Die **Nadel** oben rechts an einem Feld hält seine Verbindungen hervorgehoben, während du woanders hinsiehst.
- **Rechtsklick** auf ein Feld für alles, was seine Karte im Setup bietet: starten oder stoppen, Details, installieren, anmelden, Modelle, aktualisieren, entfernen und mehr.
- ▶ im Feld eines Agenten startet ihn in einem Terminal im Fenster.
- Die Agenten bleiben beim Scrollen immer in der Mitte des sichtbaren Teils der Karte.
- Der aktive Weg jedes Agenten — Agent → genutzter Anbieter → sein Modell — ist fett gezeichnet. Während ein Agent arbeitet, läuft Licht an dieser Linie entlang.
- Die **Nutzungsstatistik-Box**: Während ein Agent arbeitet, stehen kleine Zahlen entlang seiner Linie zum Anbieter, in der Farbe der Linie: wie schnell seine letzte Antwort kam (**t/s**, Token pro Sekunde), seine Kontextgröße (**1M**) und wie viel davon belegt ist (**%**, gelb ab 75 %, rot über 90 %, wie bei den Batterien). Sie stammen aus den eigenen Sitzungsdateien des Agenten (nur Zahlen; Claude Code, Codex und Pi). Was sie zeigt und ihre Textgröße stehen in den Einstellungen.
- Eine rote gepunktete Linie verbindet einen Agenten mit einem Anbieter, für den er eingerichtet ist, den er aber gerade nicht nutzt (kein Modell davon in Gebrauch).
- Nutzen Agenten verschiedene Modelle eines lokalen LLMs, sind die Linien zu diesen Modellen violett: Es hält sie gleichzeitig geladen oder wechselt zwischen ihnen (siehe Modelle gleichzeitig).
- Das Feld eines Online-Agenten hat dieselbe Batterie wie seine Karte im Setup: was von seinen Limits übrig ist; das Feld eines laufenden lokalen LLMs den noch freien GPU-Speicher.
- Ein pro Token bezahlter Agent — mit einem API-Schlüssel angemeldet, also ohne Plan-Limits für eine Batterie — zeigt stattdessen, was seine heute aktiven Sitzungen zu API-Preisen kosten, als kleines **„3,40 $ heute“** auf Kasten und Karte: eine Schätzung aus seinen Token-Zahlen und den Preisen, die sein Adapter nennt (Claude Code), jede Minute neu.
- Nutzt ein Programm, das nicht auf der Karte ist, ein lokales LLM, wird die Linie von diesem LLM zum Modell fett in einer eigenen Farbe (Lila oder Cyan, je nachdem, was sich von den anderen Farben des Themes abhebt), und Licht läuft daran entlang. Siehe Von anderen Apps genutzt.
- Hat ein Agent einen anderen gestartet — etwa Claude Code über seine Shell `pi -p …` —, läuft links an der Karte der Agenten ein **Pfeil** vom nutzenden zum genutzten Agenten, und Licht läuft daran entlang; sein Tooltip nennt beide.
- Das Aktualisieren-Symbol (`F5`) in der Kopfzeile sieht erneut nach, was installiert, eingerichtet und angemeldet ist, und führt die Prüfungen eines Starts noch einmal aus (neue Apps, Add-ons, Updates, offene Probleme).
- Bis die erste Linie gezogen ist, zeigt eine Karte über der Karte in drei Schritten, wie es geht; **So geht's ansehen** spielt die Animation aus der Hilfe ab. Nach der ersten Verbindung oder mit × verschwindet sie für immer.
- Zoomen mit **− 100 % +**, `Strg`+Scrollen oder `Strg`+`+` / `-` / `0`.

> **TIPP** In einem schmalen Fenster (unter etwa 1300 Pixeln, etwa der halbe Bildschirm) scrollt die Karte seitwärts.

3.4 Offene Probleme

Wenn etwas zu beheben ist, erscheint unter den Symbolen der Kopfzeile ein rotes **!**; sein Tooltip sagt, wie viele, zum Beispiel *2 offene Probleme · 4 Vorschläge*. Ein Klick öffnet **Offene Probleme**: je eine Zeile — ein Agent, der nicht angemeldet ist, ein lokales LLM ohne Modelle, eine Verbindung, die nicht funktioniert, wenig freier Platz für Modelle — mit einem Knopf dorthin, wo es behoben wird (die Details des Agenten oder lokalen LLMs, die Karte mit dieser Linie hervorgehoben, Setup, Aufräumen). Dieselben Probleme zeigen sich als rotes **!** auf ihren Karten im Setup. Tipps und was funktioniert stehen auf den Karten und Detailseiten selbst, und wofür das Setup taugt, steht im Setup (**Was du damit machen kannst**). Die Prüfung liest nur; sie ändert, startet oder sendet nie etwas, und sie zeigt nichts, solange die Karte noch leer ist.

3.5 Updates und Vorschläge

Unter den Problemen auf derselben Seite listet **Updates und Vorschläge**, was es zu sagen gibt: neue Programme oder Skills auf diesem PC, was auf diesem PC, aber nicht auf der Karte ist, und Updates für installierte Agenten und lokale LLMs, jeweils mit seinem Knopf (**MCP-Server hinzufügen…**, **Mein Setup**, **Aktualisieren**). Gibt es nur diese und kein Problem, zeigt sich das **!** der Kopfzeile in der Akzentfarbe. Nach Updates wird beim Öffnen des Fensters im Hintergrund gesucht, höchstens alle sechs Stunden. Die Suche selbst installiert nichts; ein Update beginnt erst, wenn du auf **Aktualisieren** klickst.

Ist Einstellungen › **Tägliche KI-Prüfungen** an, stehen hier auch die MCP-Server-Updates, die ein Agent im Web gefunden hat, mit **Release-Seite** und **Mit … erledigen**; sobald die neue Version auf diesem PC ist, bietet die Zeile stattdessen **Verwenden…** an (siehe Gefundene Updates).

KAPITEL 4

Agenten

4.1 Details eines Agenten

Jeder Coding-Agent hat eine Karte im Setup, installierte mit ihrer Version. Klicke die Karte eines installierten Agenten an (ASTERISM oder ··· › **Details**), oder Rechtsklick auf sein Feld auf der Karte › **Details**. Oben sagt eine Zeile, ob der Agent bereit ist (und wenn nicht, warum) und ob er gerade läuft oder wann er zuletzt benutzt wurde. Darunter ist jeder Bereich eine Karte, die sich zuklappt: Ein Klick auf ihre Überschrift öffnet oder schließt sie. Sie beginnen zugeklappt, jede mit einer einzeiligen Zusammenfassung und Abzeichen; ein Bereich, den du öffnest, bleibt auch auf den nächsten Detailseiten offen.

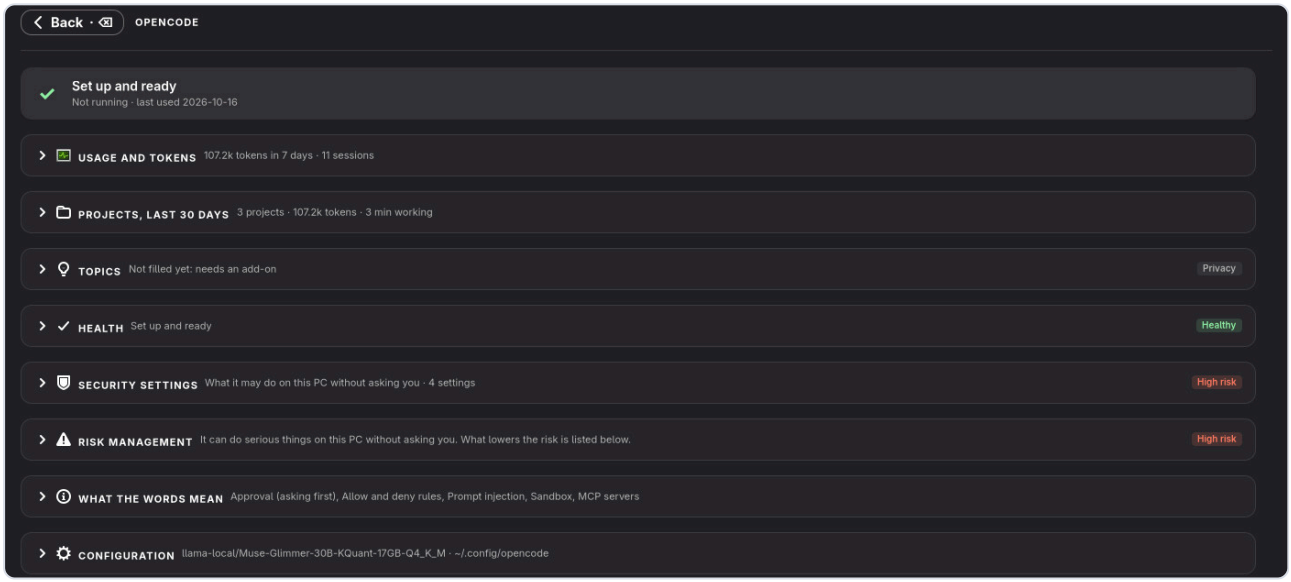


Abbildung 5 · Die Details eines Agenten: jeder Bereich zugeklappt, mit Zusammenfassung und Abzeichen.

Karte	Was sie zeigt
Nutzung und Tokens	Soweit der Agent sie aufzeichnet: Tokens und Sitzungen von heute und den letzten sieben Tagen, das Abo, ein Diagramm der Tokens pro Tag, Limits mit Zurücksetzzeiten und Tokens nach Modell.
Projekte, letzte 30 Tage	Pro Projektordner: Arbeitszeit, Sitzungen, Tokens, Subagenten-Läufe, geänderte Zeilen und der API-Wert.
Themen (Datenschutz)	Worum es in den Sitzungen ging. Das geht über Zahlen hinaus, darum ist sie mit **Datenschutz** markiert und wird nur von einem Add-on gefüllt (siehe unten); das Add-on benennt die Karte, und ihr Abzeichen sagt, wie viel es liest.
Zustand	Jede Einrichtungsprüfung mit einem ✓, oder was als Nächstes zu tun ist: anmelden, sein lokales LLM installieren oder starten, ein Modell herunterladen, MCP-Server oder Skills hinzufügen.

Karte	Was sie zeigt
Sicherheitseinstellungen, **Risikomanagement**, **Was die Begriffe bedeuten**	Was der Agent tun darf, ohne dich zu fragen, und was das Risiko senkt (siehe Gesundheits- und Sicherheitscheck).
Konfiguration	Die Einstellungen des Agenten, MCP-Server, Skills und sein Einstellungsordner.

Projekte werden für Claude Code, OpenCode und Hermes gezeigt: Arbeitszeit, Sitzungen, Tokens, Subagenten-Läufe, geänderte Zeilen und was die Tokens zu API-Preisen kosten würden. Die Zahlen stammen aus den eigenen Sitzungsdateien oder der Datenbank jedes Agenten; Text wird nicht gelesen.

ASTERISM Mit dem Add-on **Insights** (siehe Add-ons) zeigt die Karte **Themen** zuerst, welche Themen in den letzten 30 Tagen die meisten Tokens gebraucht haben (**Die meisten Tokens**: Sitzungen mit demselben Titel sind ein Thema, jedes mit seinem Anteil an allen Tokens), dann die Titel, die der Agent seinen Sitzungen selbst gegeben hat, nach Projekt (**Nach Projekt**), und bietet **Mit Claude zusammenfassen…** (wenn Claude Code installiert ist). Ihre Tokens stammen aus den eigenen Sitzungsdateien des Agenten ohne Cache-Lesevorgänge und können daher weit unter denen in **Nutzung und Tokens** liegen, das Cache-Lesevorgänge mitzählen kann. Bevor etwas gesendet wird, siehst du genau, was gesendet wird: die Titel und Zahlen, nie die Gespräche.

4.2 Agenten starten und stoppen

- ▶ im Feld des Agenten oder **Hier starten** in seinem Menü startet den Agenten in einem Terminal im Fenster, anstelle der Karte. Dafür braucht es VTE (`vte4`).
- **Im Terminal starten** öffnet ihn in einem eigenen Terminalfenster.
- Während er im Fenster läuft: **Terminal zeigen** und **Stoppen**. Stoppen beendet den Agenten und alles, was er gestartet hat, etwa seine MCP-Server.

Haben Agenten Terminals im Fenster, zeigt eine Reiterzeile über der Karte einen Reiter pro Terminal mit einem Punkt für *arbeitet*, *läuft* oder *beendet*. Ein Klick auf einen Reiter zeigt dieses Terminal; **×** schließt ein beendetes.

Ein Agent, der irgendwo auf dem Computer offen ist, bekommt einen grünen Punkt und *läuft*, und das Feld des Modells, mit dem er läuft, sagt *in Gebrauch · <Agent>*.

4.3 Netzwerkverkehr

Kasten und Karte eines laufenden Agenten zeigen, was er empfängt und sendet (↓ rein, ↑ raus). Klick auf diese Zeile, oder auf ⇅ in der Kopfzeile für alle laufenden Agenten, um seine Netzwerkseite zu öffnen: **Jetzt**, **Die letzten 5 Minuten** als Diagramm (rein grün, raus blau), **Wohin es geht** — Online-APIs, lokale LLMs, dieser PC und dein lokales Netzwerk, jeweils mit ihrem Anteil — und **Programme**: der Agent selbst und die von ihm gestarteten Programme, etwa seine MCP-Server. Gezählt wird aus den eigenen TCP-Verbindungen des Agenten, solange die App offen ist (ohne Administratorrechte; UDP wird nicht gezählt), und es bleibt auf diesem PC.

4.4 Subagenten

Claude Code und OpenCode können Teile einer Aufgabe an **Subagenten** abgeben. Solange einer der Subagenten eines Agenten arbeitet, zeigt dessen Kasten auf der Karte neben ↑↓ ein kleines Baumzeichen; sind nur fertige gelistet, ist es abgeblendet. Ein Klick darauf öffnet die Seite **Subagenten** des Agenten: eine eigene Karte im Aufbau der großen, links die Karte des Agenten, rechts eine Karte **Subagenten**. Jeder Kasten zeigt die Aufgabe des Subagenten, seinen Typ, den Ordner seiner Sitzung und **arbeitet** oder **fertig vor … Minuten**. Auf den Linien zu den arbeitenden läuft Licht; fertige sind abgeblendet. Mit der Maus über einem Kasten steht die ganze Aufgabe.

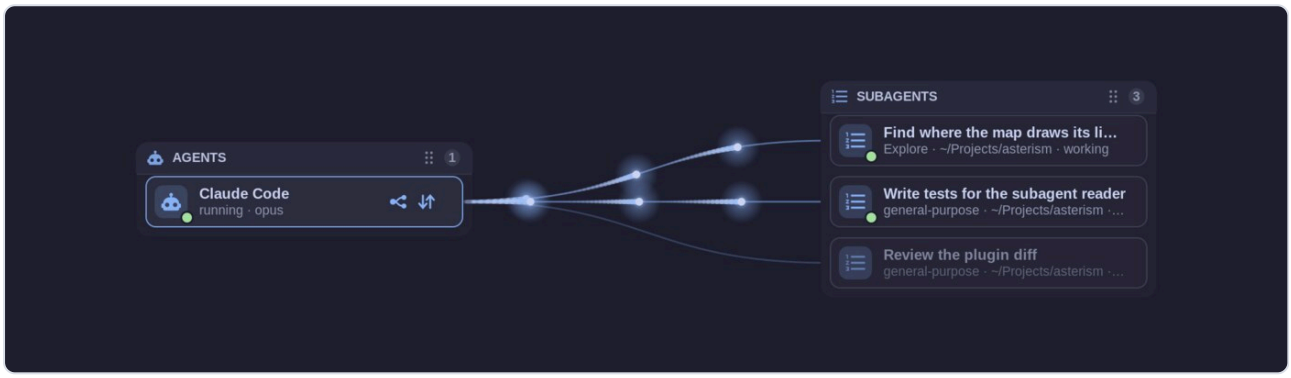


Abbildung 6 · Die Subagenten eines Agenten: zwei bei der Arbeit, einer fertig.

Wie lange fertige Subagenten gelistet bleiben (30 Minuten) und wie viele höchstens (10), steht in Einstellungen › **Subagenten**; arbeitende werden immer gezeigt. Die Seite liest nur Typ und einzeilige Aufgabenbeschreibung jedes Subagenten aus den eigenen Dateien des Agenten (den Sitzungsordnern von Claude Code, der Datenbank von OpenCode), nie, was die Subagenten sagen.

4.5 Einrichten…

Rechtsklick auf einen Agenten oder ein lokales LLM › **Einrichten…** öffnet dessen eigene Einrichtung in einem Terminal-Tab des Fensters, für das, was die App nicht selbst einstellt — zum Beispiel `hermes setup` oder Ollamas eigenes Menü; LM Studio öffnet sein Fenster. Ein Programm ohne Einrichtungsbefehl zeigt sein `--help`. Wenn es endet, sieht die Karte erneut nach und zeigt, was sich geändert hat, und einem laufenden lokalen LLM wird ein Neustart angeboten.

4.6 Anmelden

Anmelden… im Menü eines Agenten (im Setup oder auf der Karte) zeigt, was der Agent braucht, und seine eigenen Wege zur Anmeldung:

- **Ein Abo oder ein Konto** — die eigene Anmeldung des Agenten läuft in einem Terminal im Fenster (zum Beispiel `claude auth login` oder `codex login`). Dein Passwort oder die Browser-Anmeldung bleibt zwischen dir und dem Agenten; die App sieht es nie. Wo der Agent danach fragt, sagt das Terminal, was einzugeben ist (Pi und Copilot: `/login`).
- **Ein API-Schlüssel** — für Agenten, die einen lesen: Der Schlüssel kommt in ein Passwortfeld, dann direkt in den eigenen Befehl des Agenten (Codex speichert ihn selbst) oder in die Datei, die der Agent liest (Gemini: `~/.gemini/.env`, nur für dich lesbar). Er landet nie in einer Befehlszeile, einem Log oder den Einstellungen der App, und die App behält ihn nicht. Die API-Abrechnung von Claude

Code ist dessen eigene Anmeldung bei der Anthropic Console, darum geht kein Schlüssel durch die App.

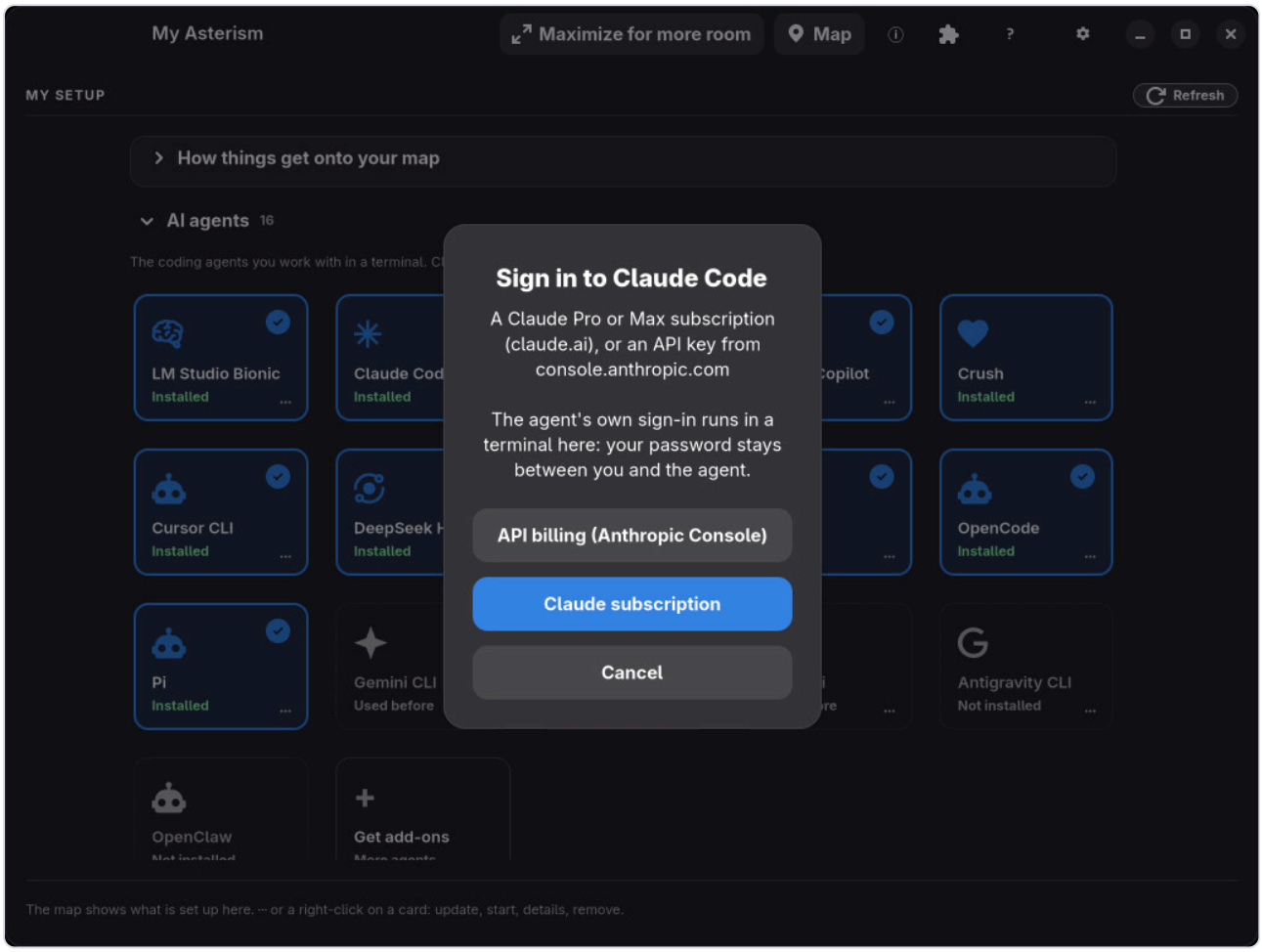


Abbildung 7 · Anmelden bei Claude Code: ein Abo oder API-Abrechnung, beide mit der eigenen Anmeldung von Claude Code.

Danach zeigt die Karte den Online-Anbieter des Agenten und seine Modelle.

4.7 Agenten aktualisieren

Gibt es eine neuere Version eines installierten Agenten, bekommt seine Karte im Setup ein gelbes ↑ **Version**, und **Updates und Vorschläge** (das **!** der Kopfzeile) nennt sie. **Aktualisieren** führt das Update so aus, wie der Agent installiert wurde — zum Beispiel `mise upgrade <tool>` oder `claude update` — in einem eigenen Terminalfenster, damit du es verfolgen kannst.

4.8 Agenten, die die App installieren kann

Drei Agenten lassen sich auf der Karte installieren, aktualisieren und deinstallieren (Rechtsklick auf den Agenten › **Installieren…**):

- **DeepSeek Harness** (`dsh` , Entwickler-Vorschau), von npm. Er kann lokale LLMs nutzen.
- **Antigravity CLI** (`agy` , Google), aus Googles Release-Manifest; der Download wird gegen seine SHA-512-Prüfsumme geprüft.

- **LM Studio Bionic** kann von hier nicht installiert werden (keine veröffentlichte Prüfsumme); hole es von lmstudio.ai. Es nutzt immer die Engine und die Modelle von LM Studio.

4.9 Neue Agenten

Taucht ein Agent auf, den die App noch nicht kennt, fragt sie einmal einen installierten Agenten (Claude Code, sonst Codex), im Web herauszufinden, wo dieser Agent seine Skills und MCP-Server aufbewahrt. Die Antwort wird vor der Verwendung geprüft, und nichts darin wird je ausgeführt. Einstellungen › **Neue Agenten nachschlagen** schaltet das aus.

KAPITEL 5

Lokale LLMs

Eine lokale LLM-Laufzeit lässt Sprachmodelle auf deiner eigenen Grafikkarte laufen, sodass Eingaben den Computer nie verlassen. Die App installiert, startet und stoppt sie für dich.

5.1 Ein lokales LLM installieren

Rechtsklick auf ein graues (nicht installiertes) lokales LLM auf der Karte › **Installieren…**. Du wählst den Programmordner, den Modellordner (vorgeschlagen wird der, den deine anderen lokalen LLMs nutzen) und ob es beim Anmelden startet. Es wird nur für deinen Benutzer installiert, mit einem Benutzerdienst, daher ist kein Passwort nötig. Jeder Download muss zur SHA-256-Prüfsumme passen, die sein Release veröffentlicht, bevor er entpackt oder ausgeführt wird.

Laufzeit	Hinweise
Ollama	Das offizielle Release (mit ROCm für AMD-Karten).
llama.cpp	Der Build deiner Wahl (Vulkan, ROCm, CUDA, CPU), als Modell-Router: jedes GGUF im Modellordner, eines gleichzeitig geladen.
LM Studio	Wird nicht von hier installiert (keine veröffentlichte Prüfsumme); hole es von lmstudio.ai. Einmal installiert, wird es gezeigt und verwaltet.
vLLM	Für NVIDIA-Karten, in einer eigenen Python-Umgebung.
Jan	Sein AppImage, entpackt, mit Jans Engine als Hintergrunddienst für Agenten.
KoboldCpp	Der offizielle Einzeldatei-Build, als Modell-Router.

So installiert, bietet ein lokales LLM außerdem **Aktualisieren…** (die vorige Version bleibt als Rückfall), **Modellordner…** und **Deinstallieren…**, das Programm, Befehl, Dienst und Einstellungen entfernt, aber nie die Modelle. Lokale LLMs, die du selbst installiert hast, werden gezeigt, aber nie verändert.

5.2 Mit den Agenten starten und stoppen

Verbinde einen Agenten einmal mit einem lokalen LLM und starte dann einfach den Agenten — auch bei geschlossener App oder nach einem Neustart. Sein lokales LLM startet mit, und wenn der letzte Agent, der es nutzt, beendet wird, stoppt das LLM etwa fünf Sekunden später, sodass sein Grafikspeicher wieder frei ist.

Das erledigt ein kleiner Hintergrunddienst, `ai-agents-map-providers.service`. Er wird mit der ersten Verbindung zu einem lokalen LLM eingerichtet und entfernt, wenn die letzte wegfällt oder wenn du Einstellungen › **Lokale LLMs mit ihren Agenten starten** ausschaltest.

> **HINWEIS** KoboldCpp braucht ein paar Sekunden zum Starten, länger als manche Agenten warten. Darum startet es beim Anmelden, und wenn der letzte Agent beendet wird, wird nur sein Modell entladen.

5.3 Details eines lokalen LLMs

Klicke die Karte eines lokalen LLMs im Setup an, oder Rechtsklick auf sein Feld auf der Karte › **Details**. Unter den Kacheln (Modelle, Agenten, Adresse) folgen dieselben zuklappbaren Karten wie bei Agenten: **Nutzung und Tokens** — was deine Agenten in den letzten Tagen darüber geschickt haben, pro Agent und Modell, aus den eigenen Sitzungszahlen der Agenten und über den Modellnamen zugeordnet —, dann Zustand, Risikomanagement, Was die Begriffe bedeuten und Konfiguration.

5.4 GPU-Speicher und GPU-Speicher freigeben

In der **GPU**-Karte über der Karte zeigt **GPU-Speicher**, wie viel belegt ist, aufgeteilt in lokale LLMs, Desktop und Apps und frei; **GPU-Aktivität** zeigt Last, Temperatur und Leistung. Das Feld jedes lokalen LLMs zeigt, wie viel Speicher es belegt, und geladene Modelle bekommen einen grünen Punkt.

Rechtsklick auf ein lokales LLM für **Starten**, **Stoppen**, Start beim Anmelden und **GPU-Speicher freigeben**: Seine Modelle werden entladen, es läuft weiter und lädt wieder eines, wenn es gefragt wird.

Ein lokales LLM, das GPU-Speicher belegt, bekommt an seinem Feld einen roten Stoppknopf. Er fragt zuerst: **GPU-Speicher freigeben** entlädt seine Modelle; **Stoppen** beendet es. Bei LM Studio heißt der Knopf **Server stoppen**: Nur sein Server endet, die App bleibt mit geladenen Modellen offen. Ein Modell, das ein lokales LLM geladen hat, das aber kein Agent nutzt, ist durch eine durchgehende helle Linie mit ihm verbunden.

LM Studio kann ohne seinen Server geöffnet sein. Sein Feld zeigt dann *Server aus* mit einem gelben Punkt, Agenten erreichen es nicht, und sein Menü bietet **Server starten** (`lms server start`).

5.5 Modelle gleichzeitig

llama.cpp, LM Studio und Ollama können mehr als ein Modell geladen halten. Verbindest du Agenten mit verschiedenen Modellen eines lokalen LLMs, stellt die App ein, wie viele es gleichzeitig geladen hält — so viele, wie seine Agenten nutzen —, mit derselben Vorschau, Sicherung und Rückgängig wie die Verbindung, und bietet einen Neustart an, wenn es läuft, denn es liest die Zahl beim Start. Wo diese Zahl nicht Sache der App ist (eine Systeminstallation oder dein eigener Startbefehl), sagt ein Hinweis, wo sie eingestellt wird.

Passen die Modelle zusammen nicht in den Speicher der GPU, sagt eine Warnung das: Das lokale LLM wechselt sie dann aus, was langsam ist, oder ein Teil eines Modells läuft auf dem Prozessor. Der Start eines Agenten prüft dasselbe: Braucht das Laden seines Modells mehr GPU-Speicher, als frei ist, wirst du vorher gefragt, mit dem, was den Speicher gerade belegt — **GPU-Speicher freigeben, dann starten**, **Trotzdem starten** oder **Abbrechen**.

5.6 Von anderen Apps genutzt

Lokale LLMs werden nicht nur von Agenten genutzt. Ein Plugin wie Video Trans, eine Chat-App oder dein eigenes Skript kann ebenfalls Anfragen schicken. Die App bemerkt das und zeigt es:



Abbildung 8 · Ein Programm, das nicht auf der Karte ist (hier `curl`), erzeugt mit llama.cpp: die cyanfarbene Linie und das Feld (Theme Tokyo Night). Der eigene Weg von Claude Code zu opus bleibt in der Akzentfarbe.

- **Auf der Karte** wird die Linie vom lokalen LLM zum Modell, mit dem es erzeugt, fett in ihrer eigenen Farbe, Licht läuft daran entlang, und das Feld des Modells bekommt Rahmen und Punkt in dieser Farbe und sagt, wer es nutzt, zum Beispiel *in Gebrauch · Video Trans*.
- **Das Rechtsklick-Menü des lokalen LLMs** und seine Seite ⓘ **Details** sagen *In Gebrauch durch Video Trans · jetzt* oder vor wie vielen Sekunden es zuletzt genutzt wurde.
- Die Legende ⓘ zeigt beide Markierungen.

Die App sieht nach, welche deiner eigenen Programme eine Verbindung zum lokalen LLM offen haben. Was sie senden, liest sie nie. Ein Plugin wird mit seinem eigenen Namen genannt, ein Agent mit seinem Namen, alles andere mit seinem Programmnamen. Die App selbst und LLM Monitor, die nur den Status lesen, zählen nicht.

> **HINWEIS** Das Licht zeigt sich nur, während das Programm erzeugt, und verblasst etwa zehn Sekunden nach seiner letzten Anfrage. Ein Programm, das offen, aber untätig ist — Video Trans ohne Sprache zum Übersetzen —, zeigt nichts. Ein Programm bleibt nach seiner letzten Anfrage eine halbe Minute im Menü aufgeführt.

KAPITEL 6

Modelle

Rechtsklick auf ein installiertes lokales LLM › **Modelle…**, oder klicke **Modell hinzufügen** auf **Setup**. Die Seite hat links eine Liste und rechts das gewählte Modell, in drei Reitern.

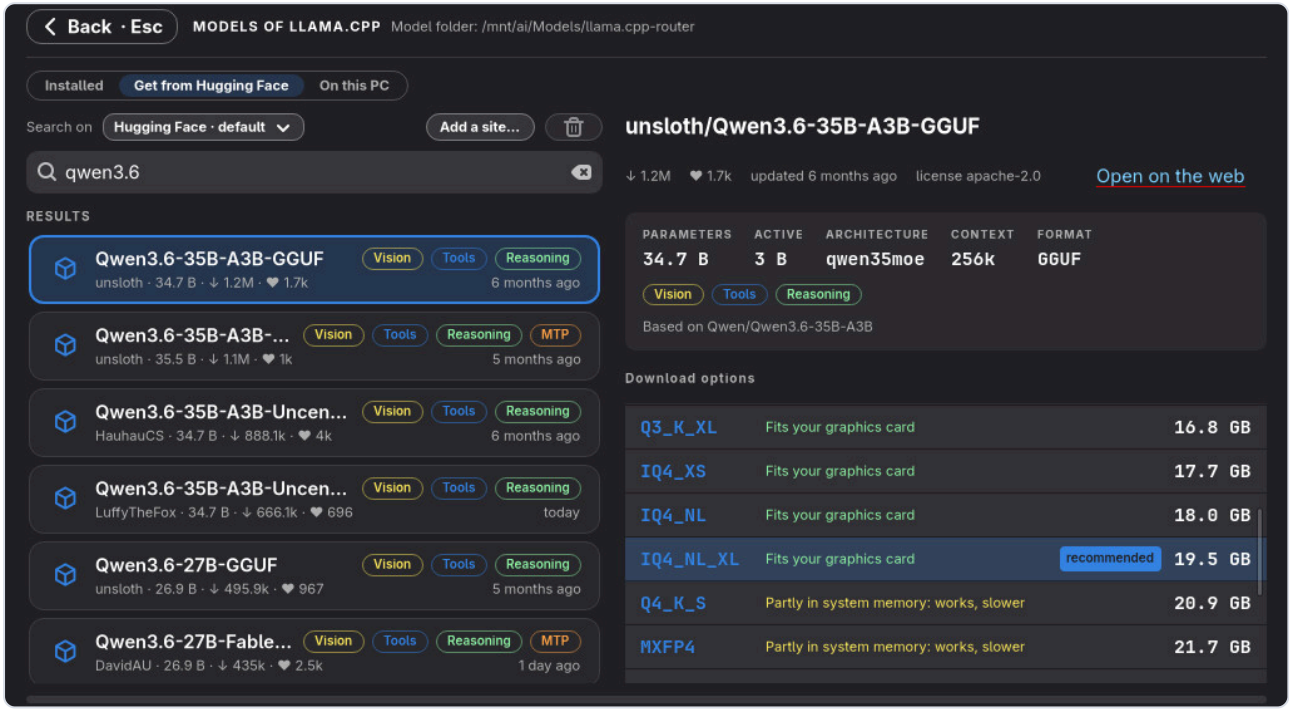


Abbildung 9 · Von Hugging Face holen: was ein Modell kann und welche Größe zu deiner Grafikkarte passt.

6.1 Von Hugging Face holen

Stöbere in den gefragten GGUF-Chatmodellen oder suche. Jedes Ergebnis zeigt Autor, Größe, Downloads, Likes, Alter und was es kann: **Bilder**, **Werkzeuge**, **Denken**, **MTP**. Die Seite des Modells zeigt Parameter, Architektur, Kontext, Lizenz und Modellkarte. **Was der Name bedeutet** erklärt die Teile des Namens, etwa *35B-A3B* oder *Q4_K_M*.

Download-Optionen listet jede Größe mit dem, wie sie passt: in die Grafikkarte, teils in den Arbeitsspeicher (funktioniert, aber langsamer) oder wahrscheinlich zu groß. Die beste Wahl ist schon getroffen; **Herunterladen** holt sie in den Modellordner.

Downloads laufen im Hintergrund und erscheinen in einem Streifen oben auf der Hauptseite. Bricht die Verbindung ab, wird es erneut versucht und der Download fortgesetzt. Wurde die App während eines Downloads geschlossen, wird er als unterbrochen aufgeführt, mit **Fortsetzen** oder **Verwerfen**.

6.2 Installiert

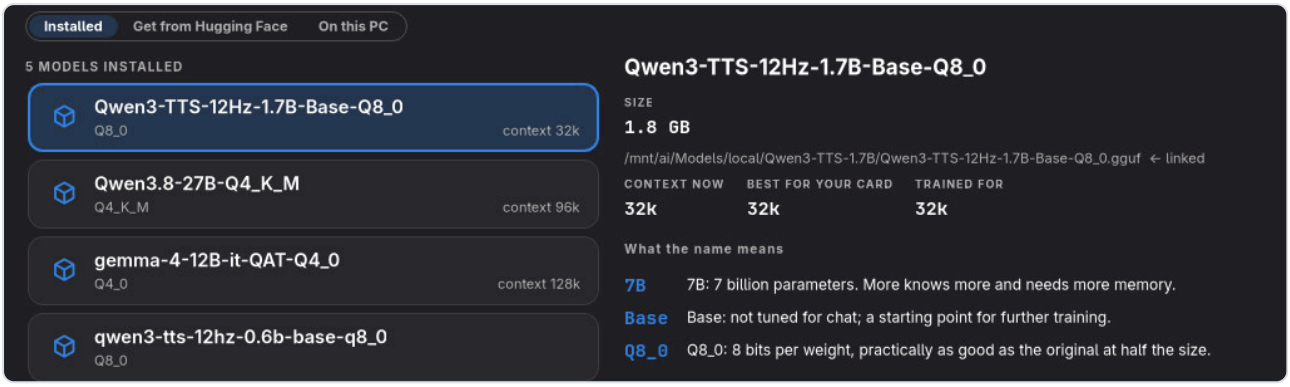


Abbildung 10 · Installiert: Kontext jetzt, am besten für deine Karte und trainiert für; was der Name bedeutet; Entfernen.

Jedes Modell zeigt Größe, Datei und Kontext — *jetzt*, *am besten für deine Karte* und *trainiert für*. **Entfernen** fragt zuerst und verschiebt die Dateien in den Papierkorb, aus dem du sie wiederherstellen kannst. Ein Modell, das nur eine Verknüpfung ist, verliert nur die Verknüpfung.

6.3 Kontextgröße

Der Kontext ist, wie viel Text das Modell auf einmal im Blick behalten kann. Die App berechnet aus der Modelldatei selbst den größten Kontext, der zu deiner Grafikkarte passt. **Kontext setzen** schreibt ihn in die Einstellungen des lokalen LLMs, mit Sicherung und Rückgängig. Agenten brauchen einen recht großen Kontext; verbindest du einen Agenten mit einem LM-Studio-Modell ohne gesetzten Kontext, wird im selben Schritt der beste gesetzt.

6.4 Auf diesem PC

Listet GGUF-Dateien, die das lokale LLM noch nicht liest — in seinem Modellordner, den Ordnern der anderen lokalen LLMs, `~/Downloads` und `~/Models`. **Hinzufügen** macht eine davon zu seinem Modell ohne zweite Kopie (über eine Verknüpfung), außer bei Ollama, das eine eigene Kopie importiert.

6.5 Modelle verschieben und löschen

Rechtsklick auf ein Modell von llama.cpp, KoboldCpp oder LM Studio › **Verschieben nach…**, oder ziehe das Modellfeld auf den Ordner eines anderen lokalen LLMs. Zuerst siehst du, wie viel verschoben wird und welcher Agent das Modell nutzt. Rückgängig verschiebt die Dateien zurück. **Löschen…** fragt zuerst und verschiebt die Dateien in den Papierkorb.

6.6 Ein Modellordner

Einstellungen › **Modellordner** legt einen Ordner für die Modelle aller lokalen LLMs fest. **Modelle aufräumen…** zeigt vorab, wie die Dateien sortiert würden (`<Ordner>/<Herausgeber>/<Modell>/`), und nichts ändert sich, bis du **Aufräumen** drückst. Mit **Alte Pfade funktionieren weiter** (standardmäßig an) bleibt an jedem alten Ort eine Verknüpfung. **Letztes Aufräumen rückgängig machen** stellt alles wieder her.

VORSICHT Beim Verschieben auf eine andere Festplatte werden die Modelle zuerst kopiert und die Originale erst entfernt, wenn die Kopie vollständig ist. Achte darauf, dass auf dem Ziel genug Platz frei ist; sonst lehnt die App das Verschieben ab.

KAPITEL 7

Verbinden durch Zeichnen

Drücke auf ein Feld der Karte und ziehe zu einem anderen, um sie zu verbinden. Jede Änderung zeigt vor dem Schreiben genau, welche Zeilen welcher Datei sich ändern, legt eine Sicherung an und lässt sich über die folgende Leiste rückgängig machen.

7.1 Agent ↔ lokales LLM

Ziehe von einem Agenten zu einem lokalen LLM oder zurück. Wähle das Modell und ob der Agent es gleich nutzt. Um eine Verbindung zu ändern, wähle ihren Agenten aus (oder klicke die Linie an): Jede Linie, die sich verschieben oder entfernen lässt, zeigt an ihrem Ende einen **orangen Griff**. Ziehe den Griff zu einem anderen lokalen LLM — oder ins Leere, um sie zu entfernen; fährst du über einen Griff, steht dort, was das Ziehen bewirkt. Ist ein Agent ausgewählt, macht ein Klick auf ein Modell eines verbundenen lokalen LLMs dieses Modell zum Modell des Agenten.

Unterstützte Agenten: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw und DeepSeek Harness.

7.2 Agent ↔ MCP-Server

Ziehe von einem Agenten zu einem MCP-Server. Die Einstellungen des Servers werden in das eigene Format dieses Agenten übernommen. Unterstützt: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode und Pi. Einstellungsdateien, die Schlüssel enthalten können, werden immer so geschrieben, dass nur du sie lesen kannst.

7.3 Agent ↔ Skill-Ordner

Ziehe von einem Agenten zu einem Skill-Ordner. Pi und Hermes bekommen den Ordner in ihre Einstellungen; jeder andere Agent bekommt pro Skill eine Verknüpfung in seinem eigenen Skill-Ordner. Ein Ordner, den ein Agent von selbst liest (die meisten lesen `~/.agents/skills`), lässt sich nicht nur diesem Agenten wegnehmen.

7.4 Wenn eine Verbindung fehlschlägt

Gleich nachdem eine Verbindung geschrieben ist, prüft die App, ob sie funktionieren wird: ob ein lokales LLM installiert ist und das Modell anbietet, ob ein MCP-Server antwortet, ob die Skill-Verknüpfungen intakt sind. Eine fehlgeschlagene Verbindung bleibt als **rote gepunktete Linie** mit einem **!** auf der Karte; eine mit nur Warnungen ist **gelb gepunktet**. Klicke die Linie oder ihr **!** an für Hinweise:

- **Erneut testen** prüft noch einmal; besteht sie, wird die Linie normal.
- **Erneut versuchen…** öffnet die Verbindungsseite für eine Verbindung, die nie geschrieben wurde.
- **Verbindung entfernen** entfernt sie.

KAPITEL 8

MCP-Server

MCP-Server geben Agenten zusätzliche Werkzeuge. **+ MCP-Server** auf der Karte oder unter den Karten öffnet die Seite zum Hinzufügen: links eine Liste, rechts die Seite des gewählten Servers.

8.1 Woher Server kommen

Reiter	Inhalt
Empfohlen	Die Referenzserver des MCP-Projekts (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) und offizielle Server bekannter Hersteller (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), nach Zweck gruppiert und auf ihre neueste Version festgelegt.
MCP-Registry	Eine Suche in der offiziellen MCP-Registry. Jedes Ergebnis nennt seinen Herausgeber so, wie die Registry ihn geprüft hat. Die Suche kann zehn Sekunden oder länger dauern. Die Ergebnisse kommen zu je 20; die Zeile unter der Liste blättert durch sie und zeigt, wo du bist, zum Beispiel **‹ Vorherige 20** · 21 – 40 / 200 · **Nächste 20 ›** (mit */ 40+*, wenn die Registry nicht sagt, wie viele es sind).
Auf diesem PC	Programme auf diesem Computer, die einen MCP-Server anbieten, gefunden durch Lesen ihrer Dateien — gestartet wird nichts. In AppImages wird nur hineingesehen, wenn du auf **Hineinsehen…** klickst.
Von Hand	Eine Befehlszeile oder eine Webadresse mit Headern.

8.2 Einen Server einrichten

Die Seite des Servers sagt, wer ihn gemacht hat und wie gut das geprüft ist, was er tut (mit Dingen, die du einen Agenten danach fragen kannst), wie er läuft — als Programm über `npx`, `uvx` oder Docker, oder als Webdienst — und was er braucht, etwa einen Ordner oder einen API-Schlüssel mit einem Link, wo es ihn gibt. Darunter **Einrichten**:

1. Fülle die Felder aus, die der Server braucht.
2. Hake die Agenten an, die ihn bekommen sollen.
3. Klicke **Testen**, um den Server nach seinen Werkzeugen zu fragen.
4. Klicke **Hinzufügen**: Alle angehakten Agenten werden auf einmal geschrieben, mit einer Vorschau und einem Rückgängig.

> **HINWEIS** Geheime Werte wie API-Schlüssel werden in der Umgebung oder in Headern übergeben, nie in einer Befehlszeile, wo andere Benutzer des Computers sie lesen könnten. Ein Server, der ein Geheimnis in seiner Befehlszeile bräuchte, muss von Hand eingerichtet werden.

8.3 Server verwalten

ⓘ **Details** auf der Karte eines MCP-Servers zeigt die Einstellungsdatei jedes Agenten, den Befehl oder die Adresse und auf Wunsch die Werkzeuge des Servers; Werte von Schlüsseln werden nie gezeigt. **Entfernen** nimmt den Server allen Agenten weg. Server bleiben in der Liste, wenn kein Agent sie mehr nutzt, damit eine Linie sie wieder verbinden kann.

Taucht in `~/Applications` eine neuere Version einer App auf, die einen MCP-Server bereitstellt, nennt sie ein Hinweis. **Aktualisieren…** entpackt die neue Version, richtet jeden Agenten darauf aus und behält die alte sieben Tage als Sicherung. **Nach Updates suchen** (unter **Auf diesem PC**) tut das nach deinem Klick für alle solchen Apps auf einmal: Es sieht in die neuen Versionen hinein, bietet alle Updates in einer Vorschau mit einer Sicherung und einem Rückgängig an und testet dann jeden aktualisierten Server; ein Hinweis nennt jeden, der nicht antwortet.

KAPITEL 9

Skills

Ein Skill ist ein Ordner mit Anweisungen, die einem Agenten eine bestimmte Aufgabe beibringen. **+ Skills** öffnet eine Seite, die wie die MCP-Seite aufgebaut ist.

9.1 Skills finden

- **Auf diesem PC**: Skill-Ordner, die Apps mitgebracht haben und die noch kein Agent liest.
- **Aus dem Web**: eine Suche im Verzeichnis skills.sh, auf einer anderen Seite mit derselben API oder in einem GitHub-Repository mit Skills. Ein Skill wird aus seinem GitHub-Repository in einen Ordner deiner Wahl geholt, standardmäßig `~/.agents/skills`.

> **VORSICHT** Skills können Skripte enthalten, die ein Agent später ausführen kann. Die Seite zeigt das Repository und listet die Skripte, bevor etwas geholt wird — sieh sie dir an.

9.2 Skill-Ordner

Jede Karte eines Skill-Ordners zeigt seine Skills und die Agenten, die ihn lesen. Ein Ordner, der über Verknüpfungen geteilt wird, erscheint einmal, dort, wo er wirklich liegt. ⓘ **Details** listet jeden Skill mit seiner Beschreibung und wie jeder Agent den Ordner bekommt.

KAPITEL 10

Einstellungen und Hilfe

10.1 Einstellungen

Das Zahnrad (`Strg`+`,`) öffnet die Einstellungen. `Rücktaste` oder `Esc` führt zurück.

Einstellung	Was sie tut
Sprache	Die Systemsprache oder English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Aussehen OMARCHY	Die Farben des Omarchy-Themes oder GNOMEs Adwaita-Aussehen mit dem hellen/dunklen Modus und der Akzentfarbe des Systems.
Karten-Animationen	Licht läuft an den Linien der Karte entlang: **Agenten und andere Programme** (die Linie eines arbeitenden Agenten und die Linie eines lokalen LLMs zu einem Modell, das ein anderes Programm nutzt), **Nur meine Agenten** oder **Aus**. Eine kleine animierte Karte unter der Einstellung zeigt, was jede Wahl bewegt.
Nutzungsstatistik-Box	Die Zahlen entlang der Linie eines arbeitenden Agenten (siehe Mit der Karte arbeiten): an oder aus, welche von **Token pro Sekunde**, **Kontextgröße** und **Kontext belegt** sie zeigt, und ihre **Textgröße** (standardmäßig 7 pt).
Subagenten	Für die Seite Subagenten: wie lange fertige Subagenten bleiben (**30 Minuten** als Standard; 10, 30, 60 oder 120) und wie viele höchstens (**10** als Standard; 5, 10, 20 oder 40). Arbeitende werden immer gezeigt.
Terminal-Tabs über der Karte	Ein Reiter pro Agenten-Terminal im Fenster.
Ecken	Wie die Fenster, abgerundet oder eckig.
Lokale LLMs mit ihren Agenten starten	Der Hintergrund-Wächter aus Mit den Agenten starten und stoppen.
Im App-Starter	Der Eintrag im Starter; aus entfernt ihn.
Agenten starten in	Der Ordner, in dem Agenten starten.
KI-Funktionen: **Neue Agenten nachschlagen**	Einen installierten Agenten fragen, wo ein unbekannter Agent seine Einstellungen aufbewahrt (standardmäßig an).
KI-Funktionen: **Tägliche KI-Prüfungen**	Einmal am Tag sucht ein Agent im Web nach neuen Versionen deiner MCP-Server (standardmäßig aus; es verbraucht die Tokens des Agenten). Siehe Smart Check.
KI-Funktionen: **Agent für KI-Prüfungen**	Claude Code oder Codex: übernimmt die KI-Prüfungen und das Nachschlagen sowie die Aufgaben **Mit … erledigen** und **Mit … beheben**.
Modellordner	Ein Ordner für alle Modelle, mit **Modelle aufräumen…**.

Einstellung	Was sie tut
Add-ons ASTERISM	**Online nach Add-ons suchen**, die Add-on-Seite und **Setup**.
Über	Die Version, der Autor und die Lizenz; **Website** öffnet das Projekt auf GitHub und **Problem melden** seine Fehlerliste.

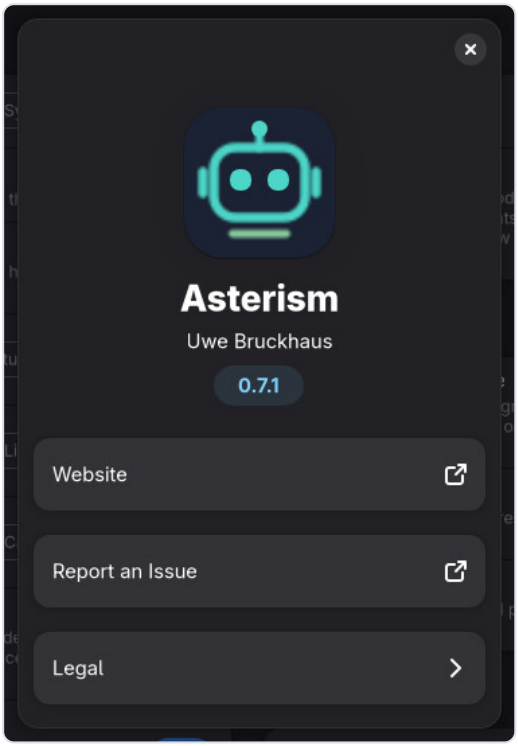


Abbildung 11 · Einstellungen › Über.

10.2 Hilfe

? (F1) öffnet die Hilfe: **Handbuch öffnen** öffnet dieses Handbuch (die Ausgabe, die mit deiner Version kam, in deiner Sprache) in deinem PDF-Betrachter; **Speicherplatz** (unten); eine Animation zum Einrichten durch Zeichnen; und LLM Monitor, ein begleitendes Leisten-Plugin, das zeigt, was ein lokales LLM tut (eine kleine Animation seines Leisteneintrags und Panels). Beide Animationen sind in den Farben deines Themes gezeichnet und folgen einem Themenwechsel sofort. Die Legende ⓘ in der Kopfzeile erklärt jeden Punkt, jede Linie und jede Markierung der Karte.

Das gefüllte **i** in der Kopfzeile öffnet **Information**: Karten für die anderen Plugins des Autors, gelesen aus dem Omarchy Plugin Marketplace, wenn die Seite öffnet, höchstens einmal pro Stunde (ohne Netzwerk die Liste, die diese Version kennt). Ein Klick auf eine Karte öffnet die Seite des Plugins im Marketplace.

10.3 Speicherplatz

Speicherplatz: Agenten, lokale LLMs und Modelle zeigt, wie viel Platz dein KI-Setup einnimmt und wo. Er wird bei jedem Öffnen der Hilfe im Hintergrund gemessen (erneut nach zehn Minuten oder mit **Neu messen**) und nur gelesen — von hier wird nichts gelöscht. Die Überschrift zeigt immer die

Summen in einer Zeile (*Agenten · Lokale LLMs · Modelle · Insgesamt*); ein Klick klappt den Abschnitt zu. **Festplatten**, **Agenten**, **Lokale LLMs** und **Modellordner** darin beginnen zugeklappt und zeigen jeweils ihre Summe und die größten Einträge; ein Klick öffnet sie.

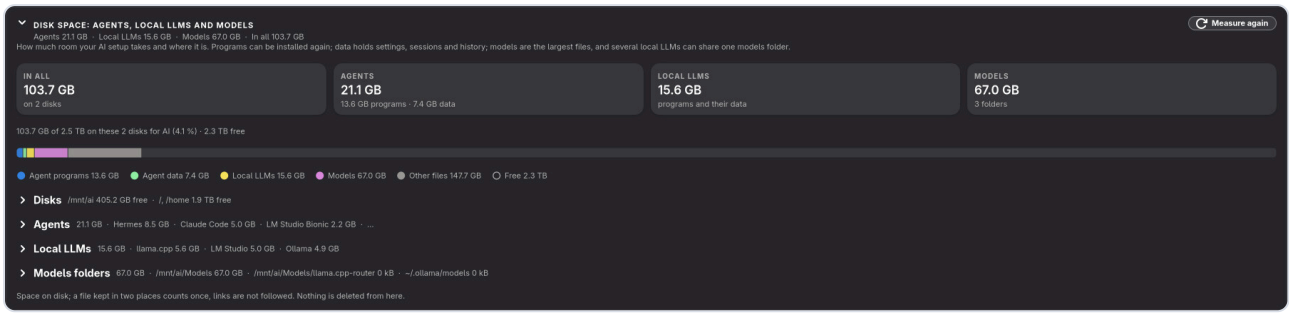


Abbildung 12 · Die Summen, der KI-Anteil am ganzen Plattenplatz und die zugeklappten Abschnitte mit ihren Summen.

- **Kacheln**: alles zusammen, dann Agentenprogramme, Agentendaten (Einstellungen, Sitzungen, Verlauf), lokale LLMs (Programme und ihre eigenen Daten) und Modelle.
- **Balken**: der ganze Platz der Festplatten, auf denen das KI-Setup liegt. Die farbigen Teile sind das KI-Setup, Grau ist alles andere auf diesen Platten, und der leere Rest ist frei; die Zeile darüber gibt den KI-Anteil in Prozent an. Zeige mit der Maus auf den Balken für genaue Größen.
- **Festplatten**: jede Platte mit dem, was das KI-Setup darauf belegt, und wie viel frei ist.
- **Agenten**, **Lokale LLMs**, **Modellordner**: je eine Zeile, die größten zuerst. Öffne eine Zeile, um jeden Ordner mit Größe und Dateianzahl zu sehen und ihn im Dateimanager zu öffnen. Ein Programmordner sagt, wie viele Versionen aufbewahrt werden; für einen Agenten mit mehreren alten Versionen in mise nennt ein Tipp `mise prune`, das die ungenutzten entfernt. Ein Modellordner sagt, welche lokalen LLMs ihn nutzen oder dass er nur Verknüpfungen zu Modellen in einem anderen Ordner enthält.

> **TIPP** Programme lassen sich neu installieren und Modelle neu herunterladen; die Daten der Agenten enthalten deine Sitzungen und Einstellungen. Wird der Platz knapp, sieh dir zuerst alte Programmversionen und Modelle an, die du nicht mehr nutzt.

Größen sind Platz auf der Festplatte: Eine an zwei Orten gehaltene Datei zählt einmal, und Verknüpfungen wird nicht gefolgt.

KAPITEL 11

Smart Check

Mit Smart Check sieht sich ein installierter Online-Agent — Claude Code oder Codex — das KI-Setup dieses PCs an, findet Updates für deine MCP-Server und erledigt Korrekturen für dich, und du siehst, was das an Tokens gekostet hat. Die zwei Funkelsterne in der Kopfzeile öffnen es; sie zeigen sich nur, solange einer dieser Agenten installiert ist. Einstellungen › **Agent für KI-Prüfungen** wählt, welcher.

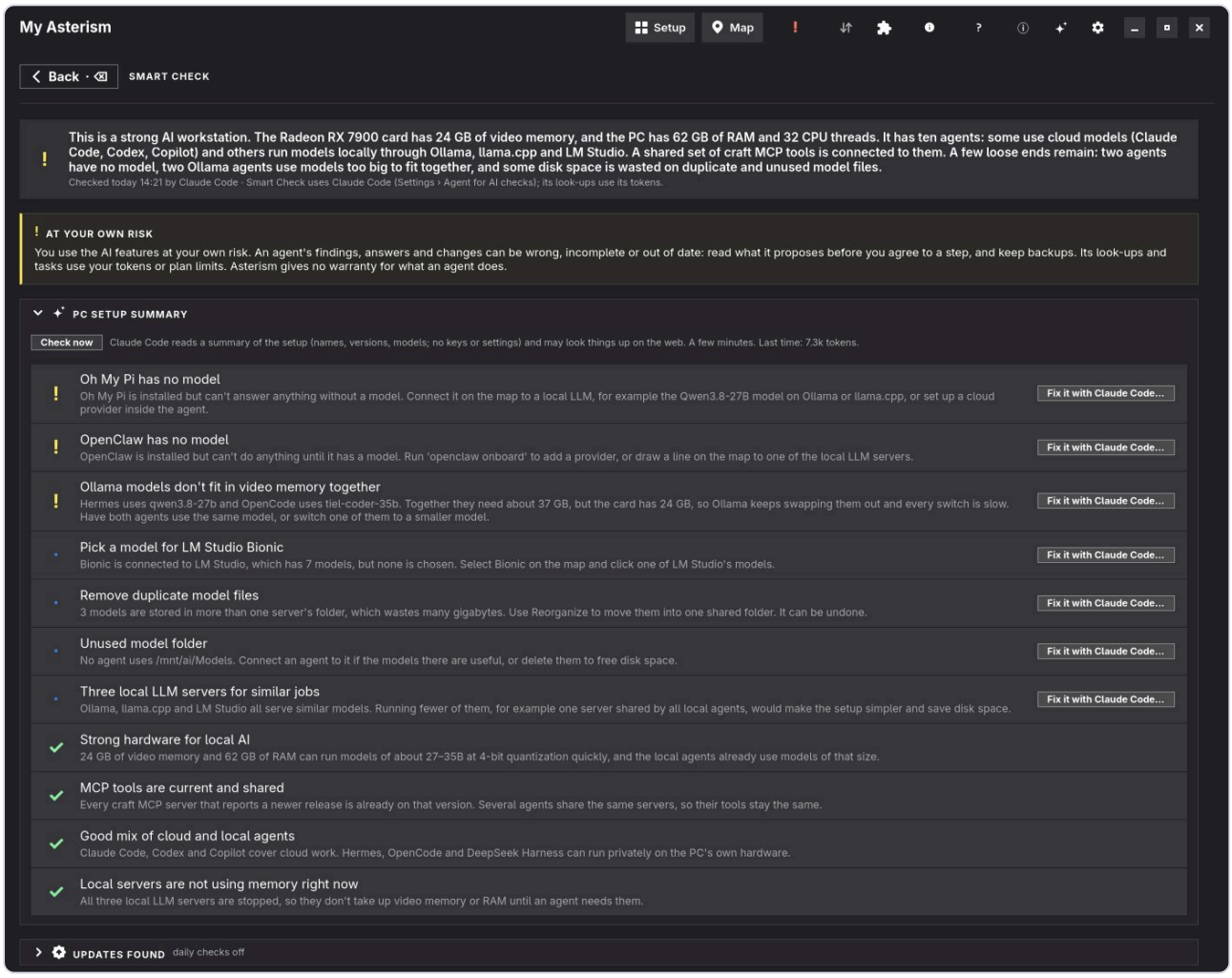


Abbildung 13 · Smart Check nach einer Prüfung durch Claude Code: Probleme (!), Tipps (·) und was gut ist (✓).

> **AUF EIGENE GEFAHR** Die Befunde, Antworten und Änderungen eines Agenten können falsch, unvollständig oder veraltet sein. Lies, was er vorschlägt, bevor du einem Schritt zustimmst, und halte Sicherungen bereit. Sein Nachschlagen und seine Aufgaben verbrauchen deine Tokens oder Plan-Limits. Die App übernimmt keine Gewähr für das, was ein Agent tut. Derselbe Hinweis steht über Einstellungen › **KI-Funktionen**.

11.1 Zusammenfassung des PC-Setups

Jetzt prüfen gibt dem Agenten einen kurzen Text über das Setup — Grafikkarte und Speicher, die Agenten mit ihren Modellen, die lokalen LLMs, die MCP-Server mit ihren Versionen, die Skills und was Offene Probleme gefunden hat — und lässt ihn Dinge im Web nachschlagen. Schlüssel, Header oder Einstellungswerte stehen nicht darin. Das dauert im Hintergrund eine Minute oder länger; die Zeile unter dem Knopf sagt, wie viele Tokens die letzte Prüfung verbraucht hat. Die Antwort ist eine kurze Zusammenfassung und eine Liste von Befunden: **!** Probleme, · Tipps und ✓ was gut ist. Antworten, die nicht standhalten, werden weggelassen.

Mit … beheben neben einem Problem oder Tipp öffnet den Agenten in einem Terminal-Tab im Fenster, mit diesem Befund als Aufgabe: Er sieht nach, ob es so ist, erklärt es und fragt dich vor jedem Schritt, der etwas ändert. MCP-Server auf eine neue Version umzustellen bleibt dem **Aktualisieren** der App überlassen (unten), das die Einstellungen aller Agenten auf einmal ändert, mit Sicherung und Rückgängig.

11.2 Gefundene Updates

Neue Versionen deiner MCP-Server, aus zwei Quellen:

- **Die täglichen KI-Prüfungen** (Einstellungen › **Tägliche KI-Prüfungen**, standardmäßig aus): Einmal am Tag sucht der Agent auf den offiziellen Release-Seiten nach neueren Versionen von Servern, deren Version die App erkennen kann — Apps, entpackt in `~/Applications/<app>-<version>/`, und npm- oder PyPI-Pakete mit fester Version. **Jetzt prüfen** auf dieser Karte fragt sofort, auch solange die täglichen Prüfungen aus sind. Jeder Fund hat **Release-Seite** und **Mit … erledigen**: Der Agent lädt das neue AppImage in einem Terminal-Tab herunter und fragt vor jedem Schritt.
- **Die eigene Suche der App**: Eine neuere Version ist schon auf diesem PC, aber ein Agent startet noch die alte (dasselbe wie das Update auf der Karte des Servers im Setup).

Sobald die neue Version auf diesem PC ist, sagt ihre Zeile das und bietet **Verwenden…** an: Die Update-Vorschau zeigt, welche Agenten auf sie umziehen; **Aktualisieren** ändert sie alle, behält die alte Version mit der Sicherung und testet danach jeden Server. Ein Server, dessen Agenten alle die neue Version starten, fällt aus der Liste.

> **TIPP** Nach einer hier gestarteten Aktion — einem Update oder einer Aufgabe in einem Terminal-Tab — bringt dich **Zurück** wieder zu Smart Check, auf dem neuesten Stand.

11.3 Token-Statistik

Heute und in den letzten 30 Tagen verbrauchte Tokens, ein Diagramm pro Tag und die Summen nach Agent und nach Plugin (andere Omarchy-Plugins, die denselben KI-Helfer nutzen, erscheinen hier auch). **API-Wert** ist, was die Läufe zu API-Preisen kosten würden, und die Zeile darunter sagt, ob dein Abo das abdeckt oder es über einen API-Schlüssel abgerechnet wird. Eine Aufgabe in einem Terminal-Tab zählt in der eigenen Sitzung ihres Agenten; die Detailseite des Agenten zeigt diese Tokens.

Darunter eine Karte pro Art von Aufgabe — die Setup-Prüfung, MCP-Server-Updates, das Nachschlagen neuer Agenten, Korrekturen — mit ihren Läufen, Tokens und den neuesten Ergebnissen.

KAPITEL 12

Gesundheits- und Sicherheitscheck

Ein Coding-Agent liest und schreibt Dateien und führt Befehle auf deinem Computer aus. Wie viel er tun darf, *ohne dich zu fragen*, steht in seinen eigenen Einstellungen, mit anderen Begriffen bei jedem Agenten. Der **Gesundheits- und Sicherheitscheck** bringt das zusammen: Auf der Detailseite jedes Agenten und jedes lokalen LLMs zeigen vier Karten, ob er funktioniert, was er darf, was das in einfachen Worten bedeutet und was das Risiko senkt.

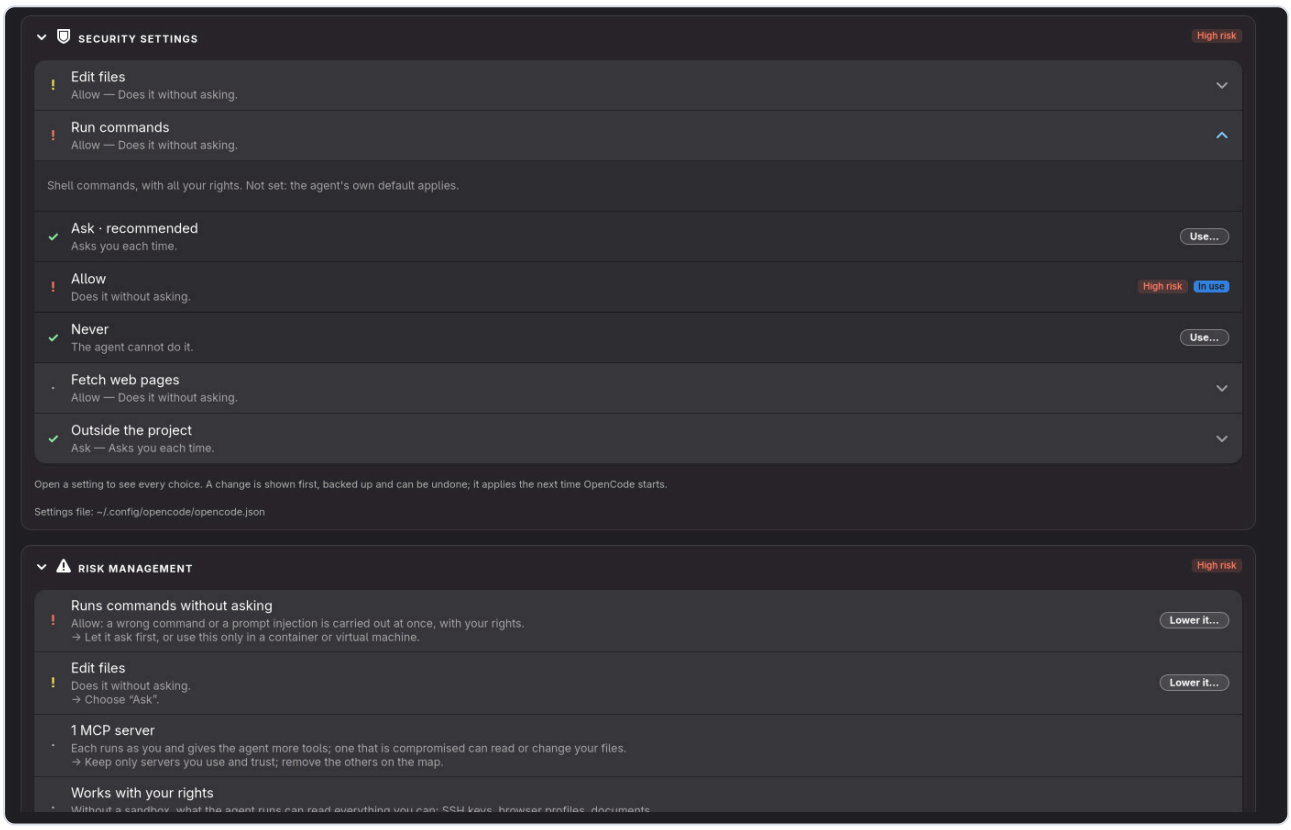


Abbildung 14 · Die Sicherheitseinstellungen von OpenCode mit geöffnetem **Befehle ausführen** und der Anfang seines Risikomanagements.

12.1 Die Karten lesen

Die Abzeichen auf den zugeklappten Karten fassen es zusammen: **Gesund**, **Bitte ansehen** oder **Nicht eingerichtet** bei Zustand, und **Geringes Risiko**, **Mittleres Risiko** oder **Hohes Risiko** bei Sicherheitseinstellungen und Risikomanagement. Öffne eine Karte für die Einzelheiten:

Karte	Was sie zeigt
Zustand	Jede Einrichtungsprüfung mit einem ✓, oder was als Nächstes zu tun ist: installiert, angemeldet, sein Modell und lokales LLM, MCP-Server und Skills.

Karte	Was sie zeigt
Sicherheitseinstellungen	Die Einstellungen des Agenten, die entscheiden, was er selbstständig tun darf, jede mit ihrem jetzigen Wert und einem Satz dazu, was das bedeutet. Das Zeichen davor nennt das Risiko: ✓ sicher, · gering, ein gelbes **!** mittel, ein rotes **!** hoch.
Risikomanagement	Was die Einstellungen zusammen bedeuten, das Ernsteste zuerst, jeweils mit dem Grund und → dem, was es senkt. Vorhandene Schutzmaßnahmen stehen mit einem ✓ da.
Was die Begriffe bedeuten	Sandbox, Freigabe, Erlauben- und Verbieten-Regeln und die anderen Begriffe der Karten, erklärt (siehe Die Begriffe erklärt).

Erlauben- und Verbieten-Regeln und vertrauenswürdige Ordner des Agenten (zum Beispiel die Liste `permissions.allow` von Claude Code) stehen unter den Einstellungen, eine Regel pro Zeile.

12.2 Eine Einstellung ändern

1. Klicke eine Einstellung an, um sie zu öffnen. Du siehst, was sie steuert, und jede Wahl mit ihrem Risiko; die genutzte ist mit **In Gebrauch** markiert, die sicherere Vorgabe mit **empfohlen**.
2. Klicke **Nutzen …** neben einer Wahl oder **Senken …** neben einem Risiko. Eine Seite zeigt die Änderung in Worten (alt → neu), das Risiko vorher und nachher und **Was sich ändert**: die Zeilen der Einstellungsdatei des Agenten.
3. Klicke **Ändern**. Die Datei wird gesichert und geschrieben; eine Meldung bietet 30 Sekunden lang **Rückgängig**. Der Agent nutzt die neue Einstellung beim nächsten Start.

> **HINWEIS** Nur der eine Schlüssel ändert sich; Kommentare und alles andere in der Datei bleiben, wie sie sind. Eine Einstellung, die mehr als eine Zeile in einer Datei braucht — etwa Hermes, das seine Befehle in Docker ausführt —, wird gezeigt und erklärt, mit dem Ort, wo sie zu ändern ist, aber nicht von hier geändert.

12.3 Was bei jedem Agenten geprüft wird

Agent	Gezeigte und änderbare Einstellungen	Datei
Claude Code	Berechtigungsmodus (Zuerst fragen, Nur planen, Nur Erlaubtes, Ohne Fragen bearbeiten, Automatische Prüfung, Alle Prüfungen umgehen), Sandbox für Befehle, Umgehungsmodus sperren; Erlauben-/Fragen-/Verbieten-Regeln	`~/.claude/settings.json`
Codex	Wann Codex fragt, Sandbox (nur lesen, Projektordner, keine), Netzwerk in der Sandbox; vertrauenswürdige Ordner	`~/.codex/config.toml`

Agent	Gezeigte und änderbare Einstellungen	Datei
GitHub Copilot	Berechtigungsmodus (Zuerst fragen, Automatische Prüfung, Alles erlauben); vertrauenswürdige Ordner, Seiten ohne Nachfrage	`~/.copilot/settings.json`
Cursor CLI	Freigabe, Sandbox, Netzwerk in der Sandbox, Websuche ohne Nachfrage; Erlauben-/Verbieten-Regeln	`~/.config/cursor/cli-config.json`
Hermes	Gefährliche Befehle (immer fragen, automatische Prüfung, nie fragen), Geheimnisse verbergen; wo Befehle laufen (nur gezeigt)	`~/.hermes/config.yaml`
OpenCode	Fragen, erlauben oder nie für Dateien bearbeiten, Befehle ausführen, Webseiten abrufen und Arbeiten außerhalb des Projekts	`~/.config/opencode/opencode.json`
Pi	Seine Werkzeuge: lesen, schreiben und Befehle ausführen, oder nur lesen. Pi fragt grundsätzlich nie, darum sind seine Werkzeuge seine Berechtigungen.	`~/.pi/agent/settings.json`

Für andere Agenten zeigt die Karte ihren Zustand und ihre MCP-Server und sagt, wo in den eigenen Einstellungen des Agenten nachzusehen ist. Für ein **lokales LLM** zeigt sie, wer es erreichen kann: nur dieser PC (`127.0.0.1` , wie es sein soll) oder jedes Gerät in deinem Netzwerk.

12.4 Das Risiko steuern

Die Risikostufe ist der ernsteste Punkt, den die Karte findet:

- **Hohes Risiko** — er kann ernste Dinge tun, ohne zu fragen: jeden Befehl ausführen oder ohne Sandbox und ohne Rückfragen arbeiten. In Ordnung in einem Wegwerf-Container oder einer virtuellen Maschine; auf deinem eigenen PC lass ihn zuerst fragen.
- **Mittleres Risiko** — manches geschieht ohne Frage, etwa das Bearbeiten von Dateien im Projekt, oder eine automatische Prüfung entscheidet für dich.
- **Geringes Risiko** — er fragt vor allem, was zählt, oder kann es gar nicht.

Gute Gewohnheiten, egal welche Einstellungen:

- **Bei Befehlen zuerst fragen** auf deinem eigenen Computer; einzelne, harmlose Befehle (etwa `git status`) mit einer Erlauben-Regel freigeben statt die ganze Shell.
- **Die Sandbox einschalten**, wo der Agent eine hat (Claude Code, Codex, Cursor CLI), und das Netzwerk darin aus lassen, solange keine Aufgabe es braucht.
- **Agenten im Projektordner starten**, nicht in deinem persönlichen Ordner, und Schlüssel und Passwörter davon fernhalten.
- **Wenige MCP-Server**: nur die, die du nutzt und denen du vertraust. Jeder läuft mit deinen Rechten.
- **Unbeaufsichtigte Arbeit** (Umgehungsmodus, „YOLO“, Pi mit allen Werkzeugen) in einem Container oder einer virtuellen Maschine laufen lassen.
- **Lokale LLMs auf** `127.0.0.1` : Ihre Server haben kein Passwort.

VORSICHT Eine automatische Prüfung (der Modus *auto* von Claude Code, *assisted* von Copilot, *auto-review* von Cursor, *smart* von Hermes) ist eine zweite KI, die jeden Schritt beurteilt. Sie spart Rückfragen, aber sie ist ein Urteil, keine Mauer.

12.5 Die Begriffe erklärt

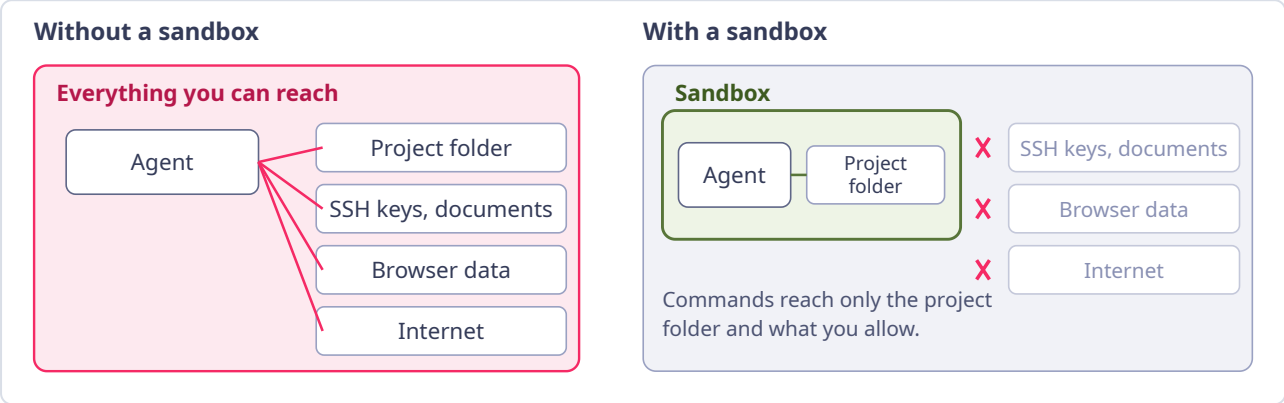


Abbildung 15 · Was eine Sandbox ändert: Befehle erreichen nur den Projektordner und was du erlaubst.

Sandbox	Ein abgeschirmter Bereich, in dem die Befehle des Agenten laufen. Darin kann ein Befehl nur den Projektordner berühren (und was du erlaubst) — nicht den Rest deines persönlichen Ordners, deine SSH-Schlüssel oder Browserdaten und oft nicht das Internet. Ohne Sandbox kann ein Befehl des Agenten alles, was du kannst.
Freigabe (zuerst fragen)	Bevor der Agent eine Datei bearbeitet oder einen Befehl ausführt, zeigt er, was er tun will, und wartet auf Ja oder Nein. Je weniger er ohne Frage darf, desto mehr Kontrolle behältst du — und desto öfter wirst du gefragt.
Erlauben- und Verbieten-Regeln	Listen von Befehlen oder Werkzeugen, die der Agent ohne Frage nutzen darf (erlauben) oder nie (verbieten). Verbieten gewinnt gegen Erlauben. Eine Regel, die jeden Shell-Befehl erlaubt, schaltet die Freigabe für die Shell aus.
Vertrauenswürdiger Ordner	Ein Ordner, in dem der Agent laut dir arbeiten darf. Dort nutzt er auch die eigenen Anweisungen und Einstellungen des Ordners, die von jemand anderem stammen können, wenn du ein Projekt klonst.
Netzwerkzugriff	Ob Befehle das Internet erreichen dürfen. Code, der deine Dateien liest und Daten verschicken kann, könnte sie preisgeben; auch das Installieren von Paketen braucht das Netzwerk.
Umgehung („YOLO“)	Alles ohne Frage und ohne Grenzen. Nur für eine Wegwerf-VM oder einen Container.
Automatische Prüfung	Ein zweites KI-Modell lässt harmlose Aktionen durch und fragt beim Rest nach.
Prompt Injection	Anweisungen, versteckt in einer Webseite, Datei oder Werkzeugantwort, die den Agenten zu etwas bringen wollen, worum du nicht gebeten hast. Zuerst fragen, eine Sandbox und kein Netzwerk stoppen sie.
Geheimnisse verbergen	Text, der wie ein Passwort, Token oder Schlüssel aussieht, wird maskiert, bevor das Modell ihn sieht oder er in einem Log landet.
Werkzeuge	Was der Agent überhaupt kann: lesen, schreiben, Befehle ausführen, im Web suchen. Ein Werkzeug, das er nicht hat, kann nicht missbraucht werden.
MCP-Server	Ein kleines Programm, das dem Agenten zusätzliche Werkzeuge gibt. Es läuft als du, mit deinen Rechten.
127.0.0.1 (localhost)	Eine Adresse, die nur Programme auf diesem PC erreichen. Ein Server auf `0.0.0.0` ist von jedem Gerät in deinem Netzwerk erreichbar.

KAPITEL 13

Add-ons

Alles, was Asterism über einen Agenten oder einen LLM-Anbieter versteht, ist ein **Adapter**: wie er gefunden und installiert wird, seine Sitzungen, Sicherheitseinstellungen, MCP-Server und Skills. In Asterism ist jeder Adapter ein **Add-on**: Die Pakete bringen alle mit, signiert und ausgeschaltet, und du schaltest die ein, die du nutzt. Add-ons bringen auch neue Agenten, neue Anbieter und Funktionen über die Zahlen hinaus, etwa **Insights** (Sitzungsthemen und eine Zusammenfassung von Claude). Das Plugin AI Agents Map hat die Adapter eingebaut und keine Add-ons.

13.1 Asterism einrichten

Ein neues Asterism öffnet sich auf seiner **Willkommensseite**. Jeder Agent und jedes lokale LLM, das Asterism kennt, ist eine Karte mit eigenem Symbol, in drei Gruppen: **KI-Agenten**, **Lokale LLMs** und **Online-Anbieter** (die Namen von Anthropic, OpenAI und den anderen auf der Karte). Jede Karte sagt, ob das Programm hier ist:

- **Installiert** (grün) — so gefunden, wie die Karte es findet; die Karte ist schon angehakt;
- **Früher benutzt** — jetzt nicht installiert, aber seine Einstellungen sind noch da;
- **Nicht installiert** — hake es an, um es danach über seine Karte zu installieren.

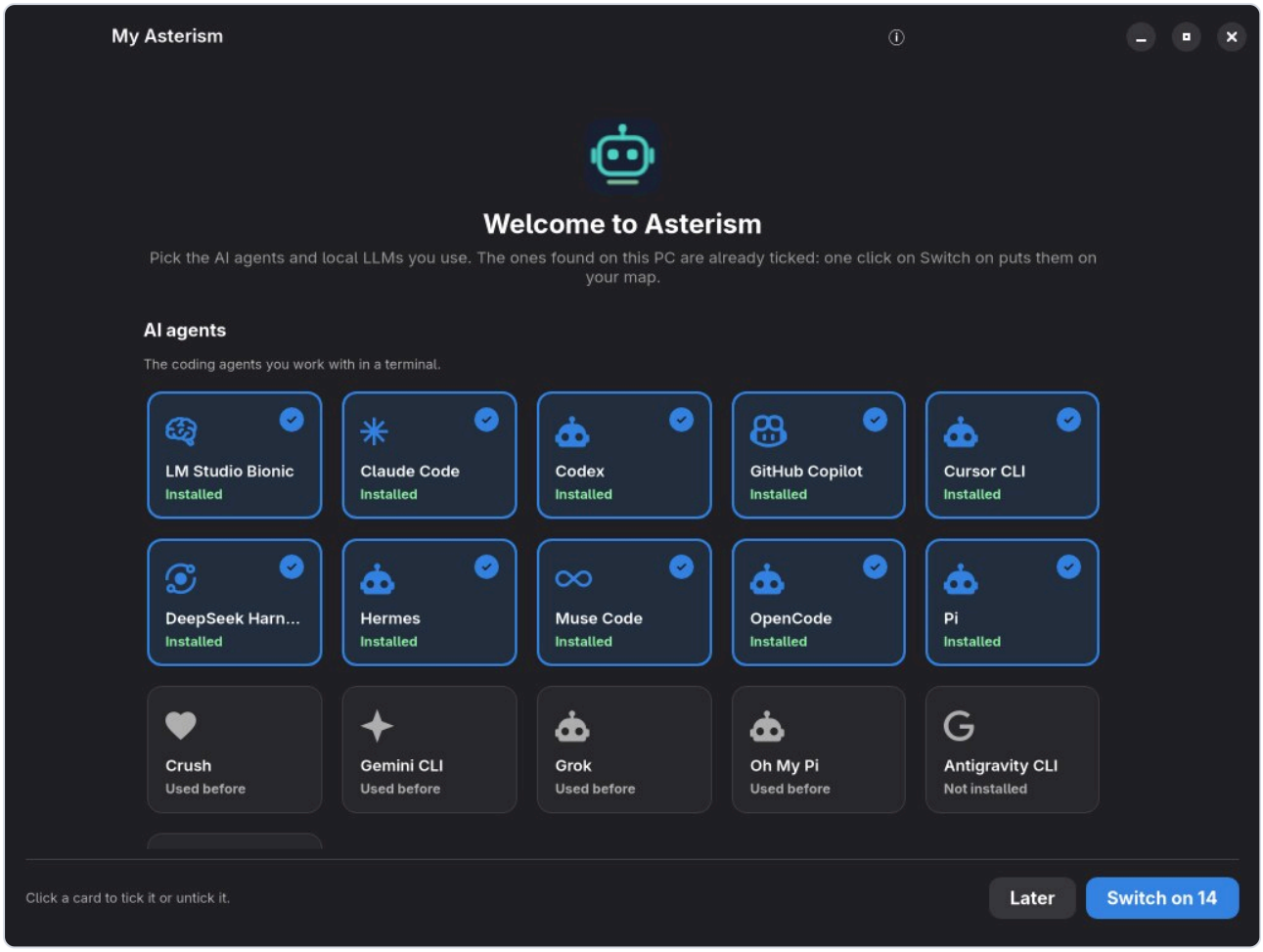


Abbildung 16 · Die Willkommensseite: gefundene Programme stehen zuerst und sind angehakt.

Klicke eine Karte an (oder drücke darauf `Leertaste`), um sie an- oder abzuhaken; eine angehakte Karte hat einen blauen Rahmen und ein Häkchen. **Einschalten** (mit der Zahl der angehakten Karten) lädt sie und zeigt die Karte; hast du Programme angehakt, die nicht installiert sind, öffnet sich **Setup**: ⋯ auf ihren Karten › **Installieren…**. **Später** startet mit einer leeren Karte. Solange nichts eingeschaltet ist, öffnet sich Asterism bei jedem Start wieder auf der Willkommensseite. Danach sind dieselben Karten im **Setup** (dem Knopf in der Kopfzeile), wo Agenten und Anbieter jederzeit ein- und ausgeschaltet werden.

13.2 Die Add-on-Seite

Öffne sie mit dem Puzzle-Knopf in der Kopfzeile, über Einstellungen › **Add-ons** oder über jeden Hinweis „Add-on verfügbar“. Sie zeigt in zuklappbaren Gruppenkarten, die offen beginnen:

- **Vorschläge für diesen PC** — ein Add-on, dessen Agent oder Anbieter hier installiert ist, aber noch nicht gezeigt wird;
- **Installierte Add-ons** — an, aus oder *nicht geladen* mit dem Grund (zum Beispiel eine geänderte Datei); Agenten und Anbieter durch eine Linie getrennt;
- **Verfügbar** — die übrigen Add-ons des Katalogs;
- **In Asterism eingebaut** — Agenten und Anbieter, die kein installiertes Add-on mitbringt (wenn du Asterism aus seinem Quellordner ohne Add-ons startest).

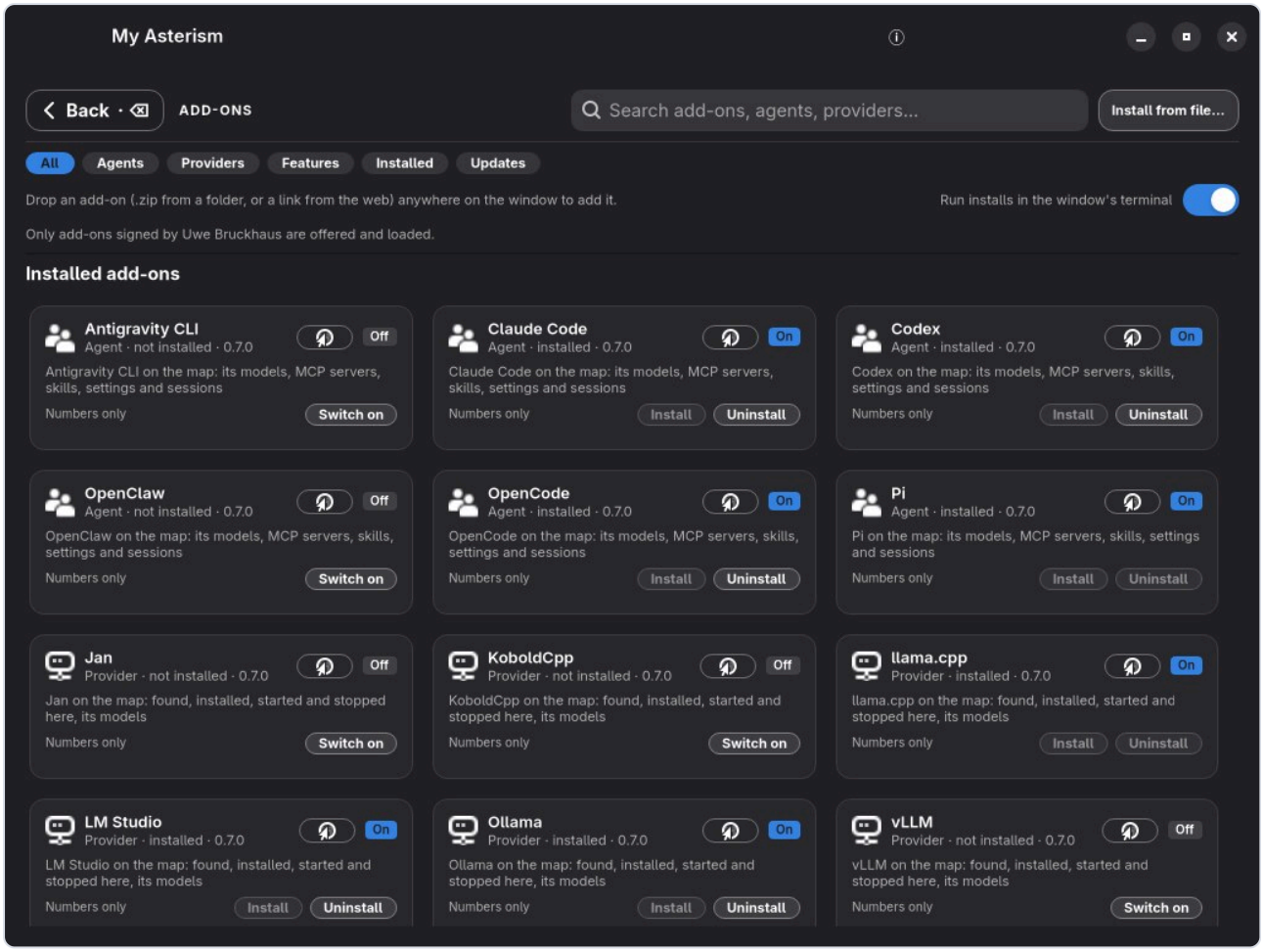


Abbildung 17 · Installierte Add-ons: an oder aus, mit Installieren und Deinstallieren für das Programm.

Die Karte eines eingeschalteten Agenten oder lokalen LLMs hat **Installieren** und **Deinstallieren** für das Programm selbst (das jeweils nicht zutreffende ist ausgegraut), und jede Karte mit einer Homepage hat einen Globus-Knopf, der sie in deinem Browser öffnet.

Die Suche findet Namen, Beschreibungen und was ein Add-on liest; die Filter darüber beschränken die Liste auf Agenten, Anbieter, Funktionen, die installierten oder die mit Update. Klicke eine Karte an für ihre Details: was es liest, was es sendet, was es darf und seine Datenschutzstufe.

13.3 Installieren, ein- und ausschalten, aktualisieren, entfernen

Installieren (oder **Add-on holen**) zeigt zuerst, was das Add-on liest, sendet und darf; erst nach deinem OK wird es heruntergeladen, geprüft und eingeschaltet. **Ausschalten** und **Einschalten** wirken sofort, ohne Neustart: Seine Agenten, Anbieter, Abschnitte und Karten erscheinen oder verschwinden überall. **Aktualisieren** ersetzt es durch die neuere Version aus dem Katalog, nachdem die neue geprüft wurde; **Entfernen** verschiebt es in den Papierkorb. **Aus Datei installieren…** nimmt ein Add-on-Zip, das du selbst heruntergeladen hast; es gelten dieselben Prüfungen.

NUR VOM AUTOR. Asterism lädt nur Add-ons, die von Uwe Bruckhaus signiert sind (Schlüssel-Fingerabdruck `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Bevor irgendetwas eines Add-ons läuft, wird die Signatur mit OpenSSHs `ssh-keygen` geprüft, jede Datei mit ihrer SHA-256-Prüfsumme verglichen, und keine Datei darf hinzugekommen sein. Ein Download muss außerdem zur SHA-256-Prüfsumme im signierten Katalog passen.

13.4 Datenschutzstufen

Stufe	Was das Add-on lesen oder senden darf
Nur Zahlen	Anzahlen, Zeiten, Tokens, geänderte Zeilen, Kosten — was Asterism ohne Add-ons zeigt.
Sitzungstitel	Die Titel, die ein Agent seinen Sitzungen selbst gegeben hat (nie die Gespräche).
Titel und eine Zusammenfassung von Claude	Diese Titel und Zahlen, an dein eigenes Claude Code gesendet — nur auf Klick, nach einer Vorschau dessen, was genau gesendet wird.

Ein Add-on kann nie mehr tun, als seine Stufe erlaubt. Einen Agenten oder einen Anbieter hinzufügen oder ein Programm installieren sind eigene Berechtigungen, die die Karte nennt.

13.5 Arten von Add-ons

Art	Was es mitbringt
Agent	Einen Coding-Agenten: wo er seine Einstellungen, Sitzungen, MCP-Server und Skills aufbewahrt, wie er installiert wird und wie er sich anmeldet. Alles andere baut Asterism daraus — die Karte im Setup, sein Feld auf der Karte, Nutzung und Tokens, Projekte, Zustand und Sicherheit.
Anbieter	Ein lokales LLM-Programm oder die Online-Anbieter: wie es gefunden und installiert wird und wie Agenten es erreichen. Asterism baut seine Karte, sein Feld auf der Karte und seine Detailseite — einschließlich **Nutzung und Tokens**: was deine Agenten darüber geschickt haben, aus ihren eigenen Sitzungszahlen.
Erweiterung	Weitere Angaben zu einem Agenten oder Anbieter, den ein anderes Add-on mitbringt, ohne ihn zu ersetzen — zum Beispiel sein Abo und seine Limits oder eine Karte auf seiner Detailseite. Sie füllt nur, was das Add-on des Agenten leer lässt; die Seite des Add-ons sagt, was es ergänzt und was davon nicht genutzt wird.
Funktion	Etwas über Agenten hinweg, etwa Insights (die Karte Themen).

Der Abschnitt eines Add-ons auf der Detailseite eines Agenten oder lokalen LLMs kommt in die Karte, um die es geht: Nutzung und Tokens, Zustand, Themen (nur bei Agenten) oder eine eigene Karte mit seinem Namen.

Ein Agenten-Add-on beschreibt seinen Agenten als Angaben (eine Datei namens `agent.toml`); die meisten bringen gar keinen Code mit, und die Seite des Add-ons sagt das: *Bringt keinen Code mit* oder, für die wenigen Agenten, die etwas brauchen (etwa ihre eigene Sitzungsdatenbank), *Angaben zum Agenten und etwas Code für das, was Angaben nicht ausdrücken können*.

13.6 Ein neuer Agent auf deinem Computer

Installierst du einen Agenten, den Asterism noch nicht kennt, sieht es im Katalog nach (Einstellungen › **Online nach Add-ons suchen**, standardmäßig an: Die Liste wird alle paar Stunden von GitHub gelesen, gesendet wird nur die Anfrage). Kennt ein Add-on diesen Agenten, sagt das **Updates und Vorschläge** (das **!** der Kopfzeile) (*„Kilo Code ist auf diesem PC, aber nicht auf der Karte“*), und eine Karte erscheint unter **Vorschläge für diesen PC**; ein Klick installiert es. Gibt es noch kein Add-on, sagen das die Details des Agenten, mit einem Link, um eines zu erbitten.

13.7 Ein Add-on erstellen

Ein neues lokales LLM oder ein neuer Agent braucht nichts als ein Add-on: Asterism baut seine Karte, seinen Kasten und seine Linien auf der Karte, seine Details, die Modelle-Seite und die Sicherheitsseite aus dem, was das Add-on darüber sagt. Wenn du ein wenig programmieren kannst, fang mit den Vorlagen im Asterism-Repository an (`templates/`, erklärt in `docs/addons.md`): eine Datei für ein lokales LLM, eine Datei mit Fakten für einen Agenten. Schick das Add-on an `github.com/uBruckhaus/asterism-addons`; ist es geprüft und signiert, erscheint es für alle unter **Verfügbar**.

KAPITEL 14

Datenschutz und Sicherheit

- **Keine Administratorrechte für die App.** Sie läuft nie als root und sieht nie ein Passwort. Einen Agenten, der als Systempaket kommt (etwa Hermes oder OpenClaw), installiert die Paketverwaltung des Systems mit `sudo` in einem Terminal, das du siehst, nachdem dir der Befehl gezeigt wurde.
- **Keine Zugangsdaten gelesen.** Sie liest die Einstellungen der Agenten, aber nie ihre Zugangsdateien. Ob ein Agent angemeldet ist, wird daran erkannt, ob eine solche Datei existiert. Werte von Schlüsseln und Headern werden nie gezeigt.
- **Keine Gespräche gelesen.** Nutzungszahlen werden nur als Zahlen und Zeiten gelesen.
- **Netzwerk nur auf Wunsch** — Modelle, MCP-Server oder Skills suchen, installieren, einen Server testen —, dazu alle paar Stunden eine Update-Prüfung.
- **Geprüfte Downloads.** Programme werden nur installiert, wenn der Download zur Prüfsumme passt, die sein Release veröffentlicht.
- **Vorschau und Rückgängig.** Jede Änderung an den Einstellungen eines anderen Programms wird vorab gezeigt, gesichert und lässt sich rückgängig machen. Einstellungen, die Schlüssel enthalten können, werden nur für dich lesbar geschrieben.
- **Begrenzt.** Jedes Programm läuft ohne Shell, und jede Antwort wird mit Größen- und Zeitgrenze gelesen.
- **KI-Funktionen nur, wenn du sie wählst.** Das Nachschlagen von Smart Check läuft auf Klick, die täglichen KI-Prüfungen nur, wenn sie eingeschaltet sind. Der Agent bekommt eine kurze Zusammenfassung (Namen, Versionen, Modelle), nie Schlüssel oder Einstellungswerte; seine Antworten werden geprüft, bevor sie gezeigt werden, und nie ausgeführt. Aufgaben laufen in einem Terminal, das du siehst, und der Agent fragt vor jedem Schritt. Du nutzt sie auf eigene Gefahr.

Die README in jedem Repository listet vollständig auf, was gelesen, gestartet und geschrieben wird.

KAPITEL 15

Problemlösung

Problem	Was zu tun ist
Die Karte ist gedrängt oder abgeschnitten.	Das Fenster ist schmaler als etwa 1300 Pixel. Mach das Fenster breiter, oder zoome heraus (`Strg`+`-`).
Ein Feld, das ich erwarte, ist nicht auf der Karte.	Die Karte zeigt nur, was eingerichtet ist: Sieh im **Setup** nach (ein ausgeschaltetes Add-on, ein MCP-Server, den kein Agent nutzt) und bei den **Zeigen**-Schaltern.
Hier starten fehlt oder tut nichts.	Installiere VTE für GTK 4 (`vte4`), oder nimm **Im Terminal starten**.
Eine Linie ist rot gepunktet.	Klicke sie an für Hinweise, behebe, was sie sagen, dann **Erneut testen**. Siehe Wenn eine Verbindung fehlschlägt.
Das lokale LLM startet nicht mit dem Agenten.	Prüfe Einstellungen › **Lokale LLMs mit ihren Agenten starten** und dass der Agent auf der Karte verbunden ist. `systemctl --user status ai-agents-map-providers` zeigt den Wächter.
Das farbige Licht von einem lokalen LLM zu seinem Modell erscheint nicht.	Es erscheint nur, während ein Programm, das nicht auf der Karte ist, mit diesem LLM erzeugt, und verblasst etwa zehn Sekunden nach seiner letzten Anfrage. Prüfe, ob die Karte sichtbar ist (ein breites Fenster), ob das Programm wirklich etwas sendet und ob Einstellungen › **Karten-Animationen** auf **Agenten und andere Programme** steht.
Der Agent antwortet schlecht oder bricht mit einem lokalen Modell ab.	Der Kontext ist vielleicht zu klein. Öffne das Modell unter **Modelle… › Installiert** und nutze **Kontext setzen** mit dem besten Wert für deine Karte.
Ein Download ist stehengeblieben.	Nach einem Verbindungsabbruch setzt er sich selbst fort. Nach einem Neustart nimm **Fortsetzen** im Streifen oben.
Ich will eine frühere Änderung rückgängig machen.	Sicherungen liegen in `~/.local/state/ai-agents-map/backups/` .

ANHANG

Referenz

A.1 Tastenkürzel

`F1`	Hilfe
`Strg`+`,`	Einstellungen
`F5`	Aktualisieren (die Karte und Setup)
`Strg`+`+` / `Strg`+`-` / `Strg`+`0`	Die Karte vergrößern, verkleinern, zurück auf 100 %
`Strg`+Scrollen	Die Karte zoomen
`Umschalt`+Klick	Zur Auswahl hinzufügen oder daraus entfernen
`Esc`	Auswahl aufheben; von einer Seite zurück
`Rücktaste`	Von einer Seite zurück

A.2 Dateien und Ordner

Pfad	Inhalt
`~/.config/asterism/` ASTERISM `~/.config/omarchy/ai-agents-map/` OMARCHY	Einstellungen, die Liste der MCP-Server, nachgeschlagene Agenten, fehlgeschlagene Verbindungen.
`~/.local/state/ai-agents-map/backups/`	Sicherungen jeder Datei, die die App geändert hat.
`~/.local/state/ai-agents-map/jobs.json`	Unterbrochene Downloads und Installationen.
`~/.local/state/ai-agents-map/reorganize.json`	Das Protokoll des letzten Aufräumens, für Rückgängig.
`~/.cache/ai-agents-map/`	Suchergebnisse, Werkzeuglisten, Update-Prüfungen, Add-on-Symbole, der letzte Smart Check (`smart-check.json`) und die MCP-Updates der KI-Prüfungen (`mcp-updates.json`).
`~/.local/state/omarchy-plugin-ai/runs.json`	Das Nachschlagen und die Aufgaben dieser App und anderer Plugins, mit ihren Tokens: die Statistik von Smart Check.
`~/.config/systemd/user/ai-agents-map-providers.service`	Der Wächter, der lokale LLMs mit ihren Agenten startet.

A.3 Glossar

Freigabe	Ein Agent fragt, bevor er eine Datei ändert oder einen Befehl ausführt (siehe Die Begriffe erklärt).
Kontext	Wie viele Tokens Text ein Modell auf einmal im Blick behalten kann.
GGUF	Das Dateiformat für Modelle, das llama.cpp, LM Studio, Ollama und KoboldCpp nutzen.
Prompt Injection	Versteckte Anweisungen in einer Seite, Datei oder Werkzeugantwort, die einen Agenten lenken wollen.
Quantisierung	Ein Modell mit weniger Bits pro Gewicht speichern (zum Beispiel *Q4_K_M*, etwa 4,8 Bit), damit es kleiner ist, bei geringem Qualitätsverlust.
Mixture of Experts	Ein Modell, bei dem für jedes Token nur ein Teil der Parameter arbeitet (*35B-A3B*: 35 Milliarden insgesamt, 3 Milliarden aktiv), was es für seine Größe schnell macht.
Sandbox	Ein abgeschirmter Bereich, in dem die Befehle eines Agenten laufen und nur den Projektordner und das Erlaubte erreichen.
Token	Ein Stück Text, ungefähr drei Viertel eines Wortes, das Modelle lesen und schreiben.
Benutzerdienst	Ein Hintergrundprogramm, das systemd nur für deinen Benutzer startet, ohne Administratorrechte.